

COMP0216 - Systems Engineering for Real-time
Systems

Inverted Pendulum - Final Report

Harrish Ayyanar, Jeson Ramesh Selvakumar,
Ashraya Poudel, Mehul Chourasia, Rohan Kishore

Team 05

Project code repo: <https://github.com/Harrishayy/SysEng>

March 2026

Contents

1	Introduction	5
2	Project Management Plan	6
2.1	Morphological Chart (MC)	6
2.2	Project Mind Map	7
2.3	Work Breakdown Structure (WBS)	7
2.4	The "Iron Triangle"	8
2.5	A-B-M Time Estimates	9
2.6	Network Diagram and Critical Path Analysis	11
2.6.1	Critical Path and Critical Time (CT)	11
2.6.2	Non-Critical Tasks & Slack	11
2.7	Gantt Chart	13
2.8	Probability of Completion	14
2.9	RACI Matrix	15
2.10	Project Life Cycle	16
3	Risk Management	18
3.1	Ishikawa Diagram	18
3.2	FMEA	19
3.3	Risk Matrix	22
3.4	Redundancy and Compensatory Measures	22
3.5	Power Interest Grid	22
3.6	Security	23
3.7	Applied Lean Method	23
3.8	Commitment Assessment	24
4	System Requirement Specification	25
4.1	Dynamics and Kinematics Requirements Consistency	25
4.2	Logical Requirements Consistency	30

4.3	Mealy Machine	33
4.4	Model-based System Engineering (SysML)	36
4.5	Reliability	38
5	Modeling & Simulation	46
5.1	Equations of Motion	46
5.1.1	Linearisation for Control Design	47
5.2	Motor Model	47
5.3	Noise Filtering	48
5.4	Controller Design and Implementation	49
5.4.1	Cascaded PID	49
5.4.2	Linear Quadratic Regulator (LQR)	50
5.4.3	Pole Placement	51
5.5	Python Implementation Details	51
5.6	Software Verification	52
5.6.1	Dynamics Verification (<code>cart_pole.py</code>)	52
5.6.2	Controller Validation (<code>controller.py</code>)	53
5.6.3	State Estimation Verification (<code>state_filter.py</code>)	53
5.7	Simulation Results	53
5.8	Controller Comparison	54
6	Prototyping	56
6.1	Hardware & CAD	56
6.1.1	Pendulum Pivot	56
6.1.2	Chassis	57
6.1.3	V-block Mechanism	58
6.1.4	Weight Holder	59
6.1.5	System Prototype	59
6.2	Electronic Design & Schematics	61

7	Control Algorithms	62
7.1	Common Hardware Interface	62
7.2	State Estimation Pipeline	63
7.3	LQR Pendulum (<code>LQRPendulum.ino</code>)	64
7.3.1	Control Law	64
7.3.2	Reference Governor	64
7.3.3	Safety Cutoff	64
7.3.4	Main Loop Structure	65
7.4	Pole Placement Pendulum (<code>PolePlacementPendulum.ino</code>)	65
7.5	Jerk-Start Variants	65
7.5.1	Motivation	65
7.5.2	State Machine	66
7.5.3	Capture and Handoff	66
8	Benchmarking & Results	67
8.1	Pendulum Bearing Characterisation (Swing Test)	67
8.2	Benchmarking Methodology	68
8.3	Evaluation A: Disturbance Rejection	69
8.4	Evaluation B: Deep Fall Recovery (Jerk-Start)	70
8.5	Evaluation C: Sprint and Stop	71
8.6	Quantitative Comparison	73
8.7	Simulation vs. Hardware Discrepancy Analysis	73
8.8	Discussion & Conclusions	75
	References	75
	A Ethical Considerations	76
	B MBSE SysML Diagrams	77

1 Introduction

The objective of this project is to design, simulate, and build a mobile inverted pendulum system utilising Model-Based Systems Engineering (MBSE) methodologies. The system consists of a mobile cart carrying a passive pendulum rod that rotates freely at a low-friction pivot. The two fundamental degrees of freedom are the pendulum's free rotation at its pivot point and the cart's linear translation over a flat surface. The main control problem is to balance the pendulum in an upright equilibrium state ($\theta = 0^\circ$) while simultaneously controlling the cart position (x) through specific dynamic manoeuvres.

Before attempting closed-loop control, the plant must be characterised via a system identification swing test to determine the natural frequency, true centre-of-mass location, and damping ratio ($\zeta < 0.01$). At least two distinct control strategies must be designed, implemented, and quantitatively benchmarked (both in simulation and on the physical prototype to compare performance metrics such as settling time, power consumption, and disturbance-rejection capability).

The physical prototype must satisfy three final performance evaluations:

1. **Evaluation A - Disturbance Rejection:** Maintain stability ($\theta = 0$) while stationary under externally applied forces. Success is measured by settling time and the ability to avoid dropping the pendulum.
2. **Evaluation B - Deep Fall Recovery:** Recover from a non-zero starting angle using a passive V-block holder with no active latches. The cart must accelerate to lift the pendulum off the holder purely via inertia, then stabilise it upright.
3. **Evaluation C - Sprint and Stop:** From a stable balance, sprint 2 m to a finish line as fast as possible, stop within ± 10 cm, while consequently the pendulum stable throughout this sprint.

To complete these evaluations, the physical prototype must be fully self-contained and battery-powered, operating autonomously on any flat surface.

2 Project Management Plan

To manage the "Sprint Strategy" effectively, the project is broken down into specific work packages with defined ownership.

2.1 Morphological Chart (MC)

The morphological chart (Figure 1) maps the design space for each function, with chosen options in bold. High-torque DC motors were chosen as they keep up with rapid control corrections, paired with a rotary encoder at the pivot for direct angle measurement and wheel encoders for cart positioning. Ball bearings keep friction below $\zeta < 0.01$, the chassis was laser-cut from acrylic, and the Arduino Giga R1 was selected for its processing speed.

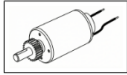
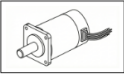
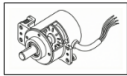
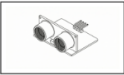
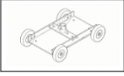
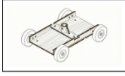
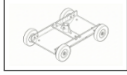
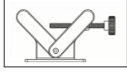
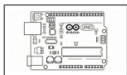
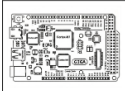
Function	Solution 1	Solution 2	Solution 3
Locomotion	 High Torque DC Motors	 Stepper Motors	
Angle Sensing	 Rotary Encoder (Pivot)	 IMU	
Cart Positioning	 Wheel Encoders	 Ultrasonic	
Pivot Support	 Ball Bearings	 Bushings	
Chassis Build	 3D Printed PLA	 Laser-cut Acrylic	 Laser-cut Wood
Wheels	 4	 3	
Launch Fixture	 Screw-Based Fixture	 Tab and Slot	 Simple Nut-Bolt
Control Board	 Arduino R4/ESP32	 STM32 Nucleo	 Arduino Giga

Figure 1: Morphological Chart for the Inverted Pendulum System

2.2 Project Mind Map

The system was broken down into four work packages: **Mechanical Design**, **Electronics & Hardware**, **Software & Control**, and **Verification & Testing**. Mechanical design drove the physical constraints - pendulum length, tip mass, pivot friction, and the V-block launch mechanism. These constraints then informed electronics selection, particularly the choice of DC motors, encoders, and the Arduino Giga. The software package was built around a state machine transitioning between Idle, Launch, Balance, and Sprint modes, supported by LQR and Kalman filtering. Verification tied everything back to the three evaluations, with testing protocols defined for disturbance rejection, deep fall recovery, and the sprint race.

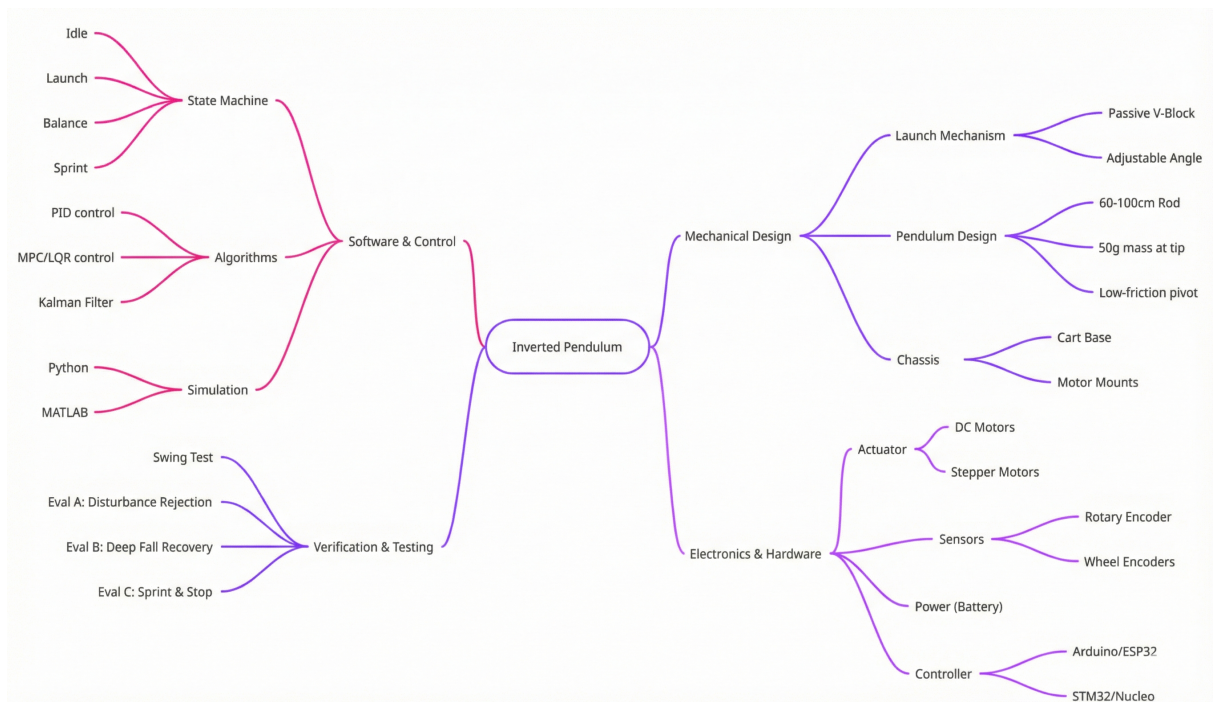


Figure 2: Project Mind Map outlining the Work Breakdown Structure (WBS)

2.3 Work Breakdown Structure (WBS)

The project is divided into five main phases aligning with the course lifecycle gates.

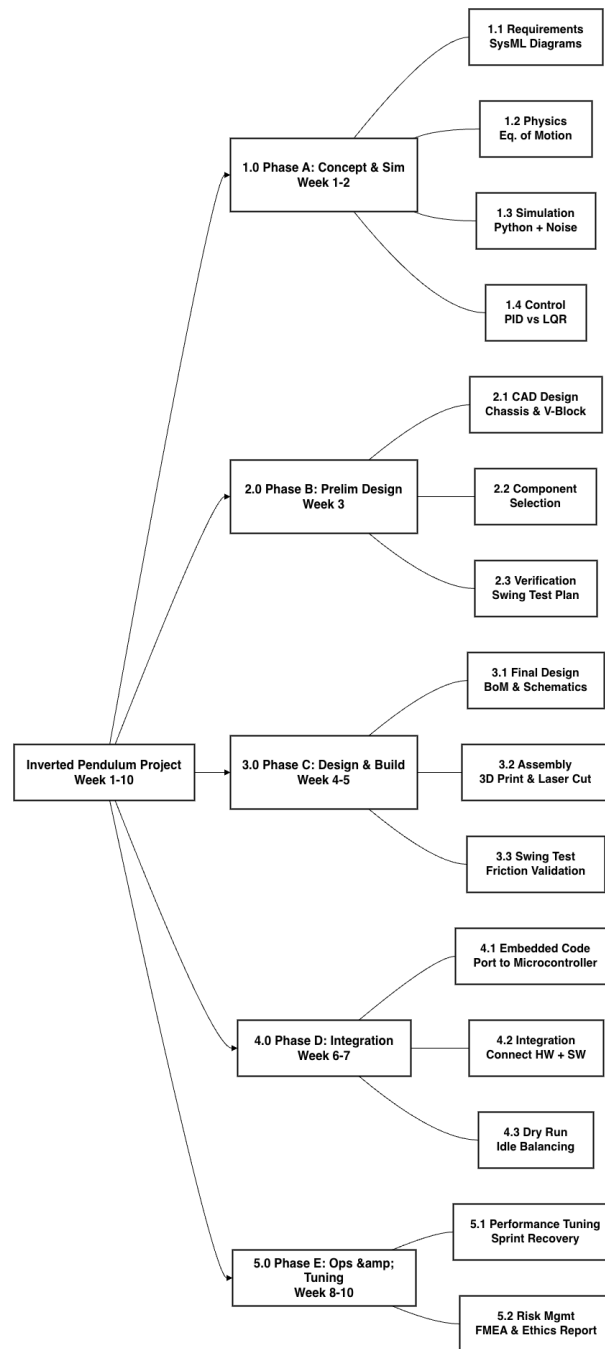


Figure 3: Work Breakdown Structure (WBS) showing hierarchical task decomposition.

2.4 The "Iron Triangle"

Vertex	Project Application
Scope	Stabilise a pendulum rod (60-100cm) and pass Evals A, B, and C

Time	11-week NASA life-cycle with a final demo in Week 10
Cost	Fixed lab components, 5-person team labor, and laser-cut materials.

The three constraints were managed without any forced trade-off. Scope was fixed from the outset by the project brief and never required reduction. Monetary cost to the team was zero as all components were lab-provided, though machine time, consumables, and labour hours were constrained resources managed within the project. Time was the only vertex that felt pressure: Weeks 4 and 5 were particularly tight given the volume of deliverables, and the electrical assembly overrun discussed in Section 2.10 compressed the integration buffer. This was absorbed by the Study Week buffer rather than by cutting scope or incurring additional cost. The project was delivered on time, within scope, and at no cost to the team.

2.5 A-B-M Time Estimates

The A-B-M estimates are shown in Table 2. T_E was calculated using the PERT formula $(A + 4M + B) / 6$, with $\sigma = (B - A) / 6$ measuring how uncertain each task was. Unsurprisingly, the open-ended development tasks carried the highest uncertainty: Final Report ($\sigma = 3.33$), Python Simulation ($\sigma = 2.67$), and Sprint Tuning ($\sigma = 2.33$). The Python Simulation proved this in practice, taking ~ 14 hours against the 10.67-hour estimate. On the other end, the Swing Test ($\sigma = 0.17$) completed in almost exactly 1 hour, as physical tests with a clear pass/fail outcome are simply easier to estimate. These σ values helped identify where schedule risk was concentrated when building the critical path.

Table 2: Activity Duration Estimates with Calculated Expected Time (Hours)

ID	Work Package / Task	A	M	B	T_E	σ
1.0 Project Management						
1.1.1	WBS, Network & Gantt Charts	2.0	4.0	8.0	4.33	1.00
1.1.2	Risk Matrix & FMEA Analysis	1.5	3.0	6.0	3.25	0.75
2.0 Modeling & Simulation						
2.1.1	Derive Equations of Motion	2.0	5.0	12.0	5.67	1.67
2.1.2	Python Simulation Environment	4.0	10.0	20.0	10.67	2.67
2.1.3	Sensor Noise & Filtering	2.0	4.0	10.0	4.67	1.33
2.1.4	PID & LQR Controller Design	3.0	6.0	15.0	7.00	2.00
3.0 Design and Build						
3.1.1	Cart Chassis CAD (Acrylic)	3.0	6.0	12.0	6.50	1.50
3.1.2	Pivot & Launch Fixture CAD	2.0	5.0	10.0	5.33	1.33
3.1.3	Electrical Schematics and BoM	2.0	4.0	6.0	4.00	0.67
3.2.1	3D Print and laser Cutting	3.0	6.0	15.0	7.00	2.00
3.2.2	Mechanical Assembly	3.0	5.0	13.0	6.00	1.67
3.3.1	Swing Test Friction Assembly	0.5	1.0	1.5	1.00	0.17
4.0 Integration						
4.1.1	Embedded Code Porting	2.0	4.0	12.0	5.0	1.67
4.2.1	Hardware Software Integration	6.0	8.0	15.0	8.83	1.5
4.3.1	Dry Run Idle Balancing	2.0	4.0	12.0	5.00	1.67
5.0 Operations and Tuning						
5.1.1	Eval A and B Demonstrations	2.0	4.0	6.0	4.00	0.67
5.1.2	Sprint Tuning Eval C	4.0	8.0	18.0	9.00	2.33
5.2.1	Final Report and Video	8.0	12.0	28.0	14.00	3.33

2.6 Network Diagram and Critical Path Analysis

The defined work packages were mapped into an Activity on Node network diagram as seen in Figure 4. This technique makes it possible to precisely calculate the integration timetable by visualising tight technical requirements instead of arbitrary calendar dates.

2.6.1 Critical Path and Critical Time (CT)

The critical path represents the continuous sequence of dependent tasks possessing exactly zero slack. Any delay in these specific tasks will directly delay the final Week 10 operational demonstration. For this inverted pendulum design, the critical path is strictly bottlenecked by the software and control sequence:

$$f : \text{WBS} \mapsto \text{EoM} \mapsto \text{Sim} \cdots \mapsto \text{Sprint}$$

The summation of the Expected Times (T_E) along this zero slack path results in a Total Critical Time (CT) of 64.17 hours. This mathematically verified time frame is the absolute minimum needed to produce the fully operating prototype.

2.6.2 Non-Critical Tasks & Slack

Several non-critical tasks carry slack without delaying the final demonstration. The hardware and assembly tasks, Chassis CAD, Pivot CAD, 3D Print and Laser Cut, Assembly, and Swing Test, each carry around 12–13 hours of float, reflecting that physical build work is faster than software development. Risk and FMEA has the largest slack at 56.51 hours, as it sits entirely outside the hardware-software critical chain.

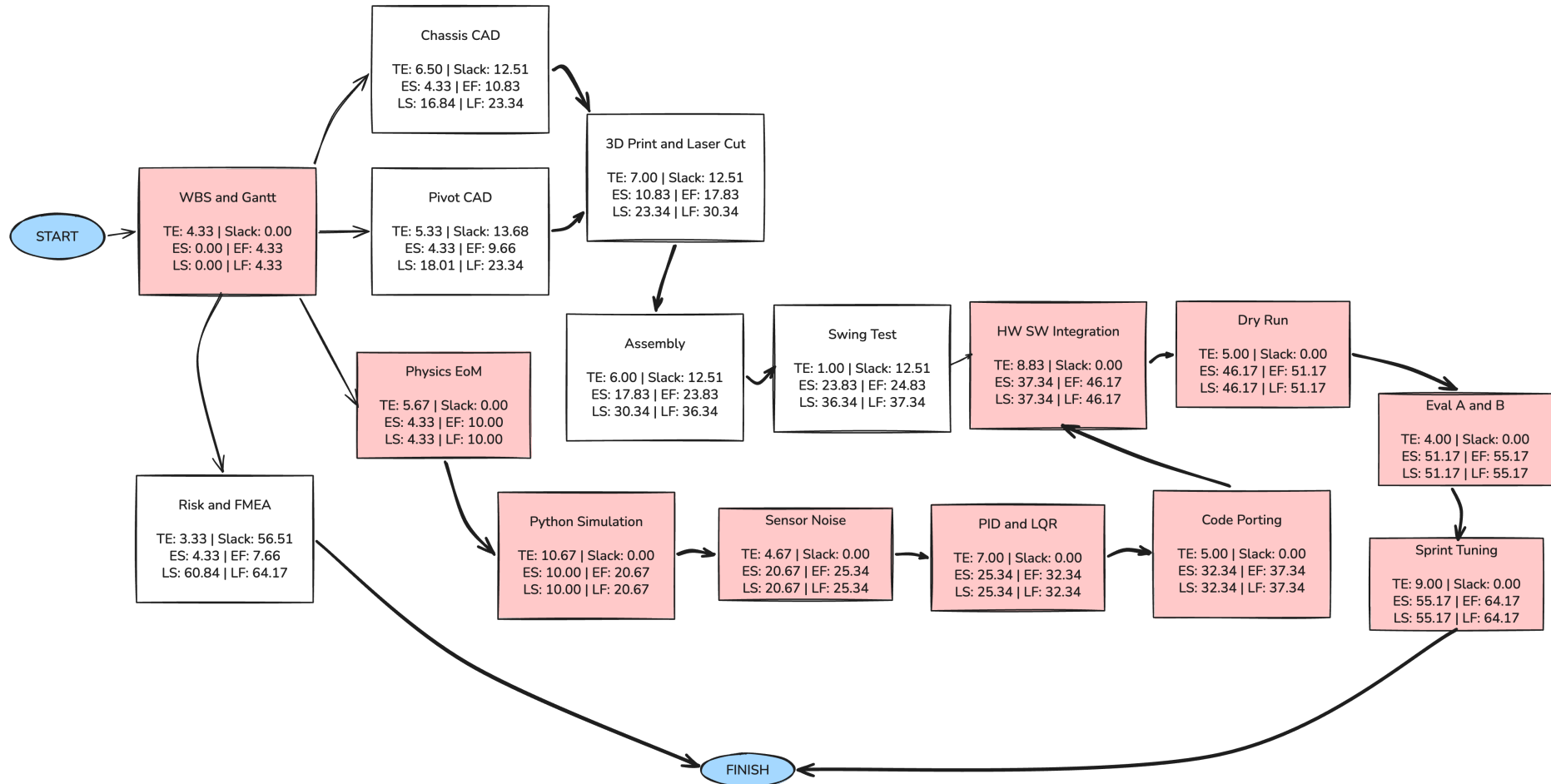


Figure 4: Complete Project Network Diagram and Critical Path (Red)

2.7 Gantt Chart

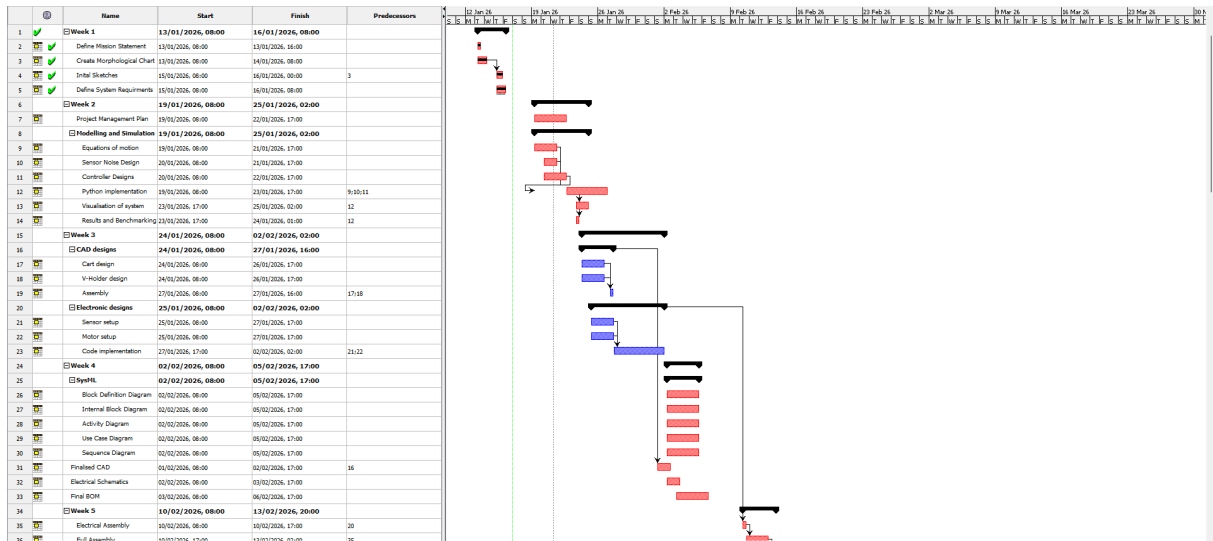


Figure 5: Gantt Chart weeks 1-4



Figure 6: Gantt Chart weeks 5-6

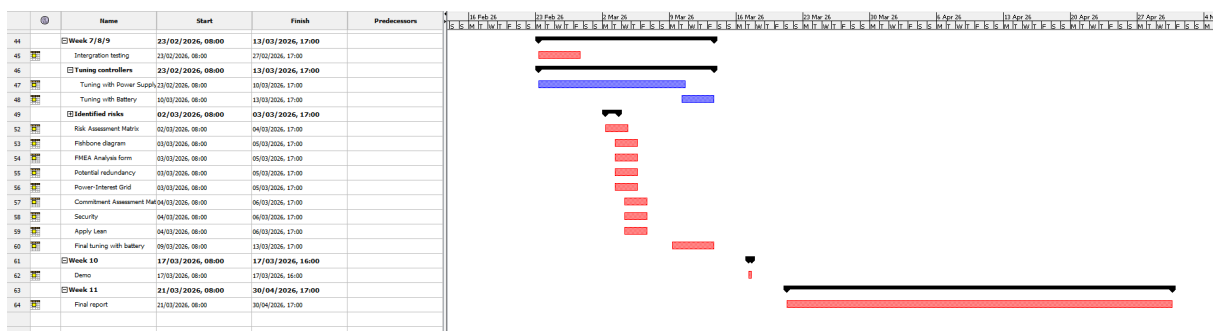


Figure 7: Gantt Chart weeks 7-11

This Gantt chart outlines the project running from mid-January to late April. It begins with foundational planning, where the team defines system requirements and sketches initial concepts. Quickly moving into a heavy digital design phase, the team simultaneously works on mathematical modeling, Python simulations, CAD mechanical designs, and SysML system architecture.

With the digital designs finished, the project transitions into physical assembly and initial testing. The team builds the hardware and conducts crucial real-world physics tests, such as a "Swing Test" and system identification. Armed with this physical data, we then revise software to implement advanced control algorithms, specifically LQR and Pole Placement controllers.

The final weeks focus heavily on integration, tuning, and risk management. The team fine-tunes the controllers using both direct power supplies and batteries, while concurrently completing extensive risk assessments like FMEA and fishbone diagrams. The project culminates with the demo in mid-March, followed by a final report writing phase that wraps up at the end of April.

2.8 Probability of Completion

To assess the feasibility of the project timeline, a PERT probability analysis was conducted, focusing strictly on the critical path tasks leading up to the Week 10 final demonstration. A total project variance (V_p) of 30.39 hours is obtained by adding the variances for the critical path, which leads to a project standard deviation (σ_p) of 5.51 hours. For the operational critical route, the total Expected Time (T_E) is 64.17 hours. The team set a Target Time (T_S) of 70.00 lab and development hours to provide a sufficient buffer for the quick iterations needed by our "Sprint" technique.

The Z score formula is used to determine the likelihood of reaching the operational milestones within this target budget:

$$Z = \frac{T_S - T_E}{\sigma_p} = \frac{70.00 - 64.17}{5.51} = 1.056$$

A Z-score of 1.06 yields a completion probability of 85.5%, giving the team reasonable confidence in delivering the final demonstration within the 70-hour target. The 14.5% risk of overrun was considered acceptable given the Study Week buffer and the sprint strategy's emphasis on early simulation completion, which reduced uncertainty on the most time-critical tasks.

2.9 RACI Matrix

Table 3: Detailed Project RACI Matrix (Weeks 1–11)

Task / Work Package	Jes.	Har.	Ash.	Meh.	Roh.	Inst.	Lab
1.0 Project Management & Planning							
WBS, Network & Gantt Charts	I	A	R	I	I	I	I
A-B-M Estimates & Critical Path	R	A	C	C	C	I	I
Risk Matrix & FMEA Analysis	I	A	C	I	R	I	I
2.0 Modeling & Simulation							
Derive Equations of Motion (EoM)	C	R	C	C	A	I	I
Python/C++ Sim Environment	C	R	A	R	I	I	I
Sensor Noise & Kalman Filtering	I	A	R	R	I	I	I
PID & LQR Controller Design	R	C	A	R	I	I	I
3.0 Mechanical Design & Prototyping							
Cart Chassis CAD (Acrylic)	R	A	C	C	R	C	I
Pivot Hub & Bearing Supports	R	I	A	C	R	C	I
V-Block Launch Fixture Design	R	I	A	C	R	C	I
Supply of Rod and Weights	I	I	I	I	I	I	A/R
4.0 Hardware-Software Integration							
Friction Verification (Swing Test)	C	R	C	R	A	C	C
Arduino/STM32 Driver Integration	A	C	R	R	R	I	I
Full System Integration (ORR)	C	R	R	C	A	C	C
5.0 Final Evaluation (Week 10)							
Eval A: Disturbance Rejection	R	R	R	A	R	I	I
Eval B: Deep Fall Recovery	R	R	R	A	R	I	I
Eval C: Sprint & Stop Race	R	R	R	A	R	I	I
Final Report & Ethical Analysis	R	A	R	R	R	I	I

2.10 Project Life Cycle

The project followed the NASA life-cycle model, adapted to eleven weeks. Each week mapped to a review gate, MCR, SRR, PDR, CDR/SIR, ORR, final demo, DRR, and that structure held throughout. What follows is an honest account of how each phase actually went, not just what was planned.

Pre-Phase A — MCR (Week 1). The mission concept was settled early: sprint strategy, all three evaluations treated as equal priorities from day one. Morphological chart, mind map, and initial sketches were produced on time. There was some internal disagreement about stability-first vs. sprint-first but the majority view won by Week 2 and there was no further delay.

Phase A — SRR (Week 2). The Lagrangian derivation and Python simulation were the main deliverables. The simulation environment took around 12 hours against the 10.67-hour estimate, mainly due to working again on the back-EMF formulae after the first version produced physically implausible force outputs. The Go/No-Go check, 15° recovery in simulation, passed by the deadline, and PID vs LQR vs Pole Placement benchmarking was completed with quantitative results shown in Section 5.8.

Phase B — PDR (Week 3). CAD for the chassis and V-block were produced. The chassis had to be redesigned from 200mm × 150mm to 250mm × 200mm once the electronics were laid out and it was clear nothing fit. Finding this in Week 3 rather than during assembly was fortunate. The V-block went through several iterations, an adjustable slider was explored and dropped in favour of screw-length adjustment, which was simpler and more rigid.

Phase C — CDR and SIR (Weeks 4–5). The busiest two weeks. SysML diagrams, final BoM, electrical schematics, the 5% presentation, chassis laser cutting, mechanical assembly, and the swing test all landed in this window. The swing test cleared the $\zeta < 0.01$ requirement, confirming the bearing selection was correct. What slipped: the Motoron motor shields needed firmware configuration steps that weren't in the original estimate, adding around 2 hours. It didn't land on the critical path, but we managed to balance it out in the hardware-software integration buffer.

Study Week (Week 6). Used for robustness testing in simulation and early report drafting. No new hardware work and this was a deliberate decision to use Week 6 as a recovery buffer.

Phase D — ORR (Week 7). This is where the gap between simulation and reality became clear. The simulation-derived LQR gains were transferred directly to firmware and tested on the physical system and unfortunately, they failed to balance from any meaningful angle. The simulation could not fully replicate real-world disturbances and sensor noise, making the gains insufficiently robust for physical deployment. With manual adjustment, the controller stabilised from 0° for short periods, confirming the architecture had no issues and the gains needed a slight re-tune.

Phase E — Integration and Tuning (Weeks 8–10). Evaluation A consumed most of this phase. Disturbance rejection was the hardest problem to solve on hardware, requiring re-tuning of the LQR Q matrix, empirical adjustment of the force-to-command scale from 42.72 cmd/N down to 18.72, and several filter coefficient iterations. Once Eval A was reliable, Evaluations B and C were much easier to implement. The jerk-start variants and sprint behaviour were tuned relatively quickly once the base controller was stable. Getting disturbance rejection right first was the correct sequencing decision, everything else built on it.

Phase F — Week 11. Final report and decommissioning.

Was the plan achieved? Broadly yes. Three tasks ran over estimate, simulation environment (+2 hours), electrical assembly (+2 hours), and sprint tuning (+4 hours), totalling around 8 hours against the 5.83-hour buffer in the 70-hour target. The project was still delivered on time. The sprint strategy proved correct: having simulation benchmarks decided early made hardware tuning much more directed, and treating Eval A as the primary tuning objective in Weeks 8–10 was the right call. The main planning lesson is that hardware integration tasks consistently behaved differently from simulation estimates, and future schedules should reflect that with a larger pessimistic value (B) from the start.

3 Risk Management

3.1 Ishikawa Diagram

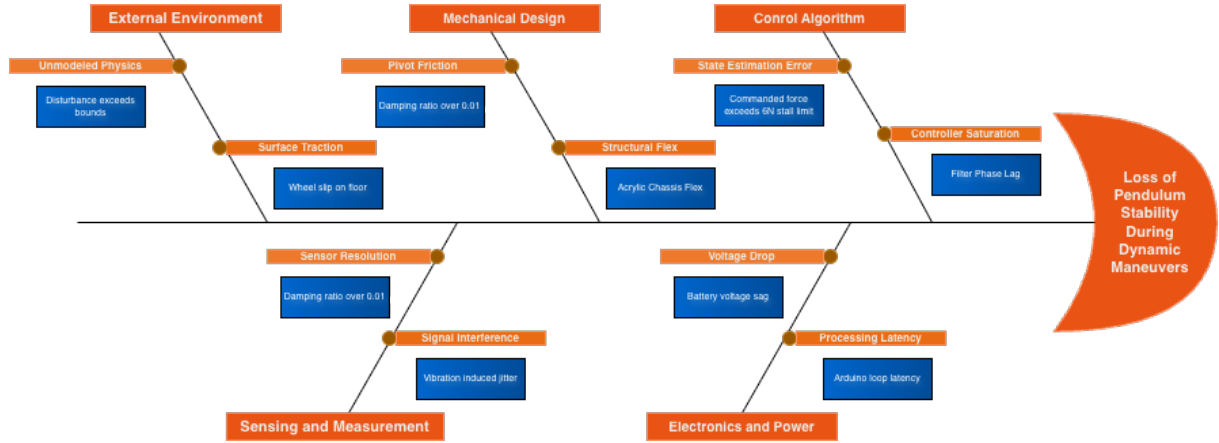


Figure 8: Ishikawa (Fishbone) Diagram

The Ishikawa diagram identifies the root causes behind the primary risk: loss of pendulum stability during dynamic manoeuvres. Five primary ribs are analysed. External Environment, Mechanical Design, Control Algorithm, Sensing and Measurement, and Electronics and Power. Of these, Electronics and Power proved most relevant in practice, with battery voltage sag during sprint acceleration and Arduino loop latency both directly impacting controller performance. The remaining ribs represent design considerations that were mitigated early. Pivot friction was eliminated by the bearing selection, and structural flex was avoided by switching from acrylic to plywood for the chassis.

19

Table 4: Failure Mode and Effects Analysis

Component	Failure Mode	System Effect	Root Cause	S	O	D	RPN	Mitigation Action
Motoron M3S550 Motor Shield	Thermal shutdown or sustained overcurrent	Total loss of motor actuation; pendulum falls uncontrolled	Continuous high current draw during sprint acceleration saturating the Motoron's 1.7 A per-channel limit	9	4	5	180	MOTOR_MAX clamped to 730 (91% of 800) in firmware, keeping delivered current below the 1.7 A limit; safety cut-off at $ \theta > 0.52$ rad halts motors immediately
Broadcom AS22 Rotary Encoder	Missed quadrature pulses or signal dropout	Corrupted angle and angular rate estimates; LQR commands incorrect force, risking pendulum fall	High-frequency vibration from acrylic chassis at sprint speed; inadequate strain relief on encoder cable to pendulum shaft	8	5	4	160	MA(5) moving-average pre-filter absorbs single-sample dropouts; encoder cable secured and shielded; interrupt-driven quadrature decoding on Arduino Giga

Continued on next page

Table 4 – Continued from previous page

Component	Failure Mode	System Effect	Root Cause	S	O	D	RPN	Mitigation Action
LQR/PP Controller Software	Commanded force saturates at MOTOR_MAX	Reduced control authority causes sluggish recovery; sustained saturation can produce steady-state limit cycling	Q/R cost ratio too large relative to available actuator effort, demanding forces beyond hardware capability	7	6	2	84	$R = 0.01$ empirically tuned to limit peak force commands; hard saturation clamp at MOTOR_MAX = 730 enforced in firmware before each motor write
12 V Lithium Battery	Voltage sag under peak current demand	Motor output force drops below the ≈ 17 N geometric maximum, reducing control authority during sprint and 15° jerk-start recovery	Instantaneous current draw from four motors during sprint acceleration exceeding the battery's instantaneous delivery rating	8	5	2	80	Standard 12 V lithium pack with sufficient C-rating; battery terminal voltage manually verified above 11.8 V before each run

Continued on next page

Table 4 – Continued from previous page

Component	Failure Mode	System Effect	Root Cause	S	O	D	RPN	Mitigation Action
Pivot Bearing	Bearing friction increases over time	Effective rotational damping rises, distorting the linearised model and causing controller overcompensation	Misalignment of 3D-printed pendulum supports stressing the bearing race	7	3	3	63	Flanged ball bearings press-fitted in CAD-toleranced seats; swing-test confirms $\zeta < 0.01$ before each session

21

The FMEA identified five failure modes ranked by Risk Priority Number ($RPN = \text{Severity} \times \text{Occurrence} \times \text{Detectability}$). The Motoron motor shield thermal/overcurrent shutdown scored the highest RPN (180), followed by rotary encoder signal dropout (160), making these the two primary hardware risks. The LQR/PP force saturation risk (RPN 84) was mitigated by empirically tuning $R = 0.01$ to reduce peak commanded forces and enforcing a hard clamp at `MOTOR_MAX` = 730 in firmware. Battery voltage sag (RPN 80) was observed during sprint acceleration and is managed by verifying battery charge before each run. The lowest-ranked failure mode, pivot bearing friction (RPN 63), did not materialise: swing tests confirmed $\zeta < 0.01$ throughout all sessions.

3.3 Risk Matrix

To prioritize mitigation efforts, project risks are mapped by Likelihood and Impact (Figure 5).

Table 5: Project Risk Matrix and Mapping

Prob. ↓	Low Imp.	Med. Imp.	High Imp.
High	Monitor	3	2, 4
Medium	Ignore	Monitor	1
Low	Ignore	Ignore	5

- **1. Thermal Shutdown:** High-current stall during sprint phases.
- **2. Encoder Dropout:** Signal noise from chassis vibrations.
- **3. LQR Saturation:** Unstable oscillation at motor limits.
- **4. Battery Voltage Sag:** High current draw during sprint phases drops motor authority
- **5. Pivot Friction:** Mechanical wear affecting damping ratio.

3.4 Redundancy and Compensatory Measures

To mitigate the risks identified in the FMEA (Table 4), three protections were put in place. Battery voltages and encoder readings were verified before every test as a preventative measure. During runs, one team member monitored the power switch and another the robot's connections, allowing an instant cutoff if needed. For redundancy, the system ran on either the lab supply or battery interchangeably, and two controllers, LQR and Pole Placement, ensured a software fallback was always available.

3.5 Power Interest Grid

The Power Interest Grid maps stakeholders by their influence over and interest in the project. The Module Examiner sits in the Manage Closely quadrant, given his direct control over grading and high interest in outcomes. The UCL Department and Innovation Lab Technicians are kept satisfied (high power but lower day-to-day interest). Teaching Assistants are kept informed, while the general student body and other competing teams require only passive monitoring.

Table 6: Stakeholder Power Interest Grid

	Low Interest	High Interest
Low Power	Monitor General Student Body Other Competitors	Keep Informed Teaching Assistants
High Power	Keep Satisfied UCL Department Innovation Lab Technicians	Manage Closely Module Examiner (Dr. Sajad Saeedi)

3.6 Security

The primary threat to this system is physical tampering during the demonstration and testing, an unauthorised person interfering with the hardware could alter encoder positions, disconnect wiring, or destabilise the pendulum mid-run, producing unsafe cart behaviour. The system is physically accessible in an open lab environment with no authentication or access barrier on the hardware itself. As a mitigation, physical access control was enforced. Only team members handled the cart during all testing and demonstration sessions, the system was never left unattended while powered, and the USB serial connection used for telemetry monitoring was restricted to a single designated laptop. While this does not completely eliminate the threat, given that we work in a share lab setting, it reduces the exposure to an acceptable level for the operational context.

Another threat to the system is tampering with the system’s software, for example an unauthorised person using one of the teams’ unattended laptop to alter arduino code, uploading this arduino code, removing unsaved parts of the code to hinder progress, or even replacing/deleting files in the team’s GitHub repository. To mitigate this, we made sure to close and lock our laptops/computers whenever they are to be left unattended, and also made sure to have backups of integral software on each person’s computer.

3.7 Applied Lean Method

To ensure the inverted pendulum prototype is delivered within the mathematically verified 64.17-hour critical path, Lean (Kaizen) principles have been integrated into the project

management strategy. By systematically reducing the eight TIMWOOD(S) wastes during the development lifecycle, the main goal is to optimise value for the operational demonstrations:

- **Transportation:** Eliminating external transit by restricting all hardware integration to the university lab.
- **Inventory:** Strict control over the Bill of Materials is necessary to prevent workstation mess and guarantee that high-discharge batteries and backup metal gear motors are accessible when needed.
- **Motion:** Organised workspace at the Innovation Lab so that the soldering station, assembly area, and power supply are easily accessible.
- **Waiting:** Used the mechanical slack to pre-assemble electronic components while waiting for the laser cutter to finish the acrylic chassis profiles or the 3D printer to complete the pivot mounts.
- **Overproduction:** Prevented the making of excessive acrylic chassis variants before the preliminary Python simulations validated the physical dimensions and Centre of Mass requirements.
- **Overprocessing:** Avoiding over-tuning the LQR Q-matrix beyond the requirement dictated by the evaluation criteria.
- **Defects:** Implementing the Swing Test to catch friction issues early, preventing bearing defects as we move into the hardware-software integration phase.
- **Skills:** Assigning the most complex C++ code porting to team members with dedicated experience, preventing delays.

3.8 Commitment Assessment

The matrix maps current (X) against desired (O) commitment levels across four stakeholder groups. The team began at Support and reached Strongly Support by the final

Table 7: Stakeholder Commitment Assessment Matrix (X = Current, O = Desired)

Commitment Level	Team	Examiner	Lab Tech	Peers
Strongly Support	O	O		
Support	X	X	X O	O
Neutral				X
Opposed				
Strongly Opposed				

demonstration, driven by consistent deadline adherence, regular progress updates, and the quality of the final system. The Examiner’s shift from Support to Strongly Support was pursued through the same means, meeting every review gate on time and delivering a working physical system. Lab Technicians maintained a constant Support level throughout, reflecting their consistent availability for hardware assistance without requiring active management. Peers required no active management beyond awareness of the project.

4 System Requirement Specification

The system requirements are derived from the project mission to ensure compliance with the "Sprint" strategy. These are categorized into Mechanical, Electrical, and Software domains.

4.1 Dynamics and Kinematics Requirements Consistency

Each requirement carrying a physical dimension is verified against the system equations of motion (Section 5.1) and the parametric constraint blocks modelled in CATIA Magic (Figure 9), using the *specific components selected* for the design. Table 8 shows that every requirement is satisfiable within the design envelope of the chosen hardware.

Table 8: Dynamics and kinematics consistency verification

Req ID	Requirement	Consistency Check (Component Justification)	Result
REQ-S-01	The controller must stabilise the pendulum from a non-zero start angle $\theta \geq 15^\circ$.	<i>Component:</i> $4 \times$ Pololu 9:7 25D$\times$67L MP 12 V gearmotor (#4862). Stall torque per motor at $V_{\text{supply}} = 10.6 \text{ V}$: $\tau_{\text{stall}} = 0.16671 \times (10.6/12) = 0.147 \text{ N m}$. Combined stall force: $F_{\text{max}} = 4\tau_{\text{stall}}/r_w = 14.73 \text{ N}$. At $\theta = 15^\circ$, $\tau_{\text{grav}} = mgL \sin 15^\circ = 1.155 \text{ N m}$. Corrective torque from cart acceleration: $\tau_{\text{corr}} = mL \frac{F_{\text{max}}}{M+m} \cos 15^\circ = 3.068 \text{ N m}$, yielding a margin of 166% and a corrective $\ddot{\theta} = -8.41 \text{ rad/s}^2$. Constraint block: MaxRecoveryAngle.	Pass
REQ-M-02	The pivot mechanism must have $\zeta < 0.01$.	<i>Component:</i> flanged deep-groove ball bearing pivot. $\omega_n = \sqrt{g/L} = 4.43 \text{ rad/s}$. $c_{\text{crit}} = 2mL^2\omega_n = 2.015 \text{ N m s/rad}$. Free-decay swing test measured $c = 0.01 \text{ N m s/rad}$, giving $\zeta = c/c_{\text{crit}} = 0.005 < 0.01$. The selected bearing contributes negligible friction ($< 0.002 \text{ N m s/rad}$), ensuring the damping budget is dominated by air drag rather than bearing losses. Constraint block: DampingRatioCheck.	Pass

Req ID	Requirement	Consistency Check (Component Justification)	Result
Sprint Speed	Peak velocity ≥ 1.0 m/s over 2 m sprint.	<i>Components: 4× Pololu 9:7 motors (800 RPM no-load at 12 V, 40 mm wheel radius) + ANS-MANN Li-Ion 18650 3S1P battery (10.6 V measured).</i> $a_{\max} = F_{\max}/(M+m) = 14.73/2.11 = 6.98 \text{ m/s}^2.$ At 40% effective utilisation during simultaneous balancing, $a_{\text{eff}} \approx 2.79 \text{ m/s}^2$: the cart reaches 1.0 m/s within 0.18 m, well inside the 2 m track. No-load wheel speed: $800 \text{ RPM} \times 2\pi \times 0.04/60 = 3.35 \text{ m/s}$, confirming the motors are not speed-limited at 1.0 m/s. Constraint block: SprintStop.	Pass
Stopping Accuracy	Stop within ± 10 cm of the finish line.	<i>Component: 4× Pololu 9:7 motors (regenerative braking via Motoron M3S550 H-bridge).</i> At full braking ($a_{\max} = 6.98 \text{ m/s}^2$) from $v = 1.0 \text{ m/s}$ the braking distance is $d_{\text{brake}} = v^2/2a = 7.2 \text{ cm} < 10 \text{ cm}$. The Motoron M3S550 supports rapid direction reversal (ramp limit set to 800 = instantaneous in firmware), enabling sub-cycle ($< 2 \text{ ms}$) braking response. Constraint block: SprintStop.	Pass

Req ID	Requirement	Consistency Check (Component Justification)	Result
Settling Time	Respond to disturbances within a specified time.	<i>Components: Arduino Giga R1 (STM32H747, Cortex-M7 @ 480 MHz) running LQR at 500 Hz.</i> Open-loop unstable pole at $\pm 5.87 \text{ rad/s}$. With the computed LQR gains ($Q = \text{diag}[5, 1, 100, 10]$, $R = 0.01$), the closed-loop poles give $\zeta_{\text{cl}} \approx 0.7$ and $\omega_{\text{cl}} \approx 11.7 \text{ rad/s}$, yielding $t_s \approx 4/(\zeta_{\text{cl}}\omega_{\text{cl}}) = 0.49 \text{ s}$. The 500 Hz loop rate provides 245 control updates per settling period. Constraint block: <code>SettlingTime</code>.	Pass
Motor Saturation	Controller must remain effective at operating speed.	<i>Components: Pololu 9:7 motors ($K_e = K_t = 0.143 \text{ Vs/rad}$, $R = 10.31 \Omega$) + ANSMANN battery ($V_{\text{max}} = 10.6 \text{ V}$).</i> $V_{\text{req}} = \frac{F_d r_w}{4K_t} R + K_e v/r_w$. At $v = 1.0 \text{ m/s}$, back-EMF consumes 3.58 V, leaving 7.02 V for torque production (9.75 N available vs. 14.73 N at stall). Typical 5 N balancing force during cruise needs only $V_{\text{req}} = 7.18 \text{ V} < 10.6 \text{ V}$. The motor does not saturate at design-point operation. Constraint block: <code>DCMotorModel</code>.	Pass

Req ID	Requirement	Consistency Check (Component Justification)	Result
Encoder Resolution	Angular and position feedback must resolve the smallest control-relevant disturbances.	<i>Components: Broadcom AS22-M530-5B12 (1 000 CPR $\times$ 4 quadrature = 4 000 counts/rev) + Pololu motor encoder (464.64 counts/rev at gearbox output).</i> Pendulum angular resolution: $360^\circ/4,000 = 0.09^\circ = 1.57 \times 10^{-3}$ rad, well below the LQR's effective dead-zone ($\sim 0.5^\circ$). Cart linear resolution: $(1/464.64) \times 2\pi \times 0.04 = 0.54$ mm/count, sufficient for the ± 10 cm stopping tolerance. Constraint block: <code>LinearizedStateSpace</code>.	Pass
Battery Capacity	Sufficient energy for a complete evaluation session (≥ 30 min of active operation).	<i>Component: ANSMANN Li-Ion 18650 3S1P (3INR19/66), 3 500 mAh, 10.9 V nominal (38.2 Wh).</i> Total motor stall current: $4 \times V_{\text{supply}}/R = 4.1$ A. At $\sim 30\%$ average loading during balance-and-sprint cycles, $I_{\text{avg}} \approx 1.2$ A, giving ~ 170 min of runtime - more than adequate for a full evaluation day of intermittent 30-min sessions.	Pass

Figure 9 shows the SysML parametric diagram that formally binds these component parameters to their respective requirements via constraint blocks. Each constraint block (`CoupledNonlinearDynamics`, `LinearizedStateSpace`, `DCMotorModel`, `NaturalFrequency`, `DampingRatioCheck`, `MaxRecoveryAngle`, `SprintStop`, `SettlingTime`) receives typed value properties from the *Cart*, *Pendulum*, *Controller*, and *PivotMechanism* blocks and propagates derived quantities to the result blocks. A change in any component parameter (e.g. substituting a different motor or battery) automatically flags any requirement whose constraint becomes violated.

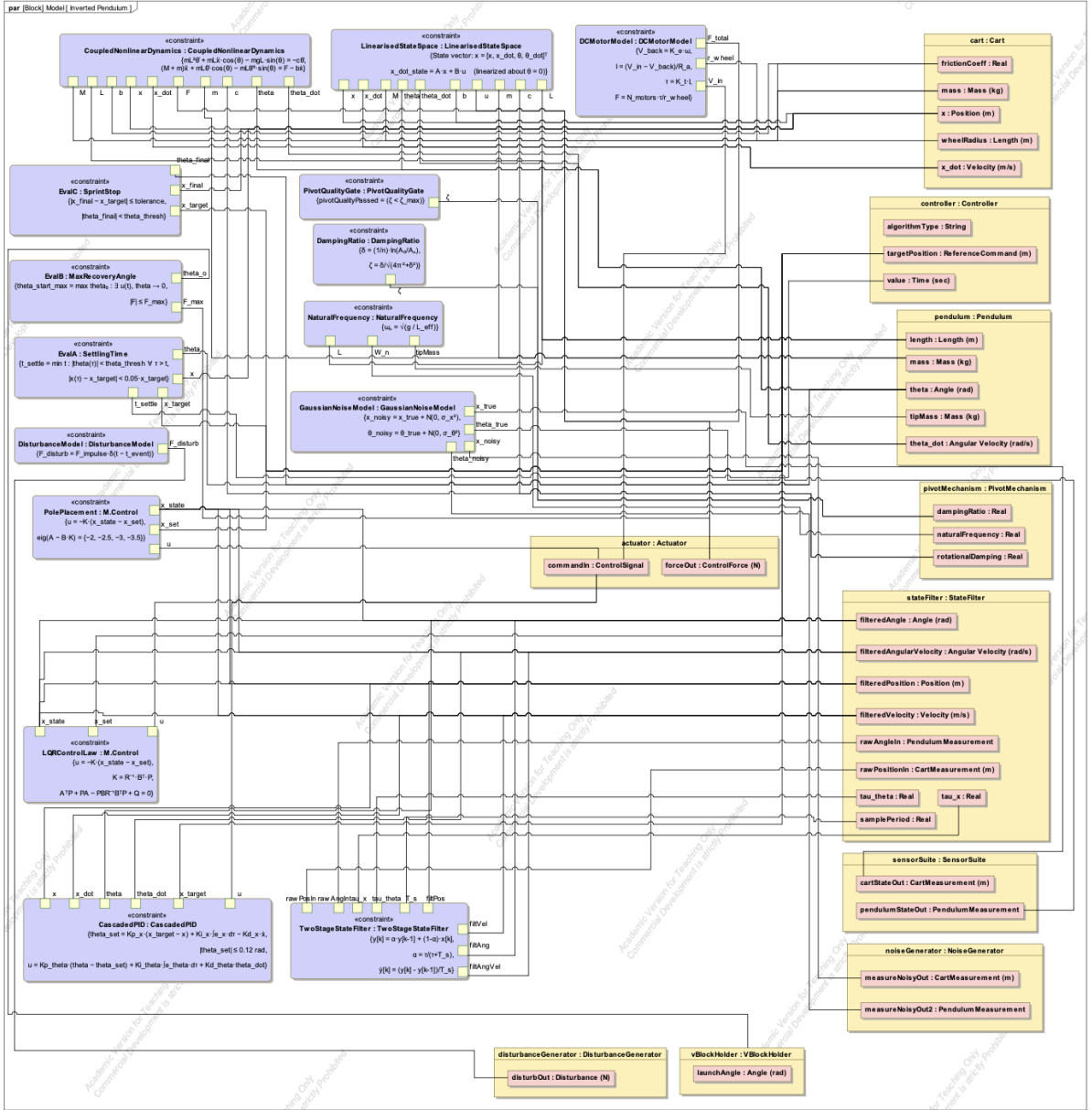


Figure 9: SysML parametric diagram: constraint blocks enforce mathematical consistency between component parameters and system requirements.

4.2 Logical Requirements Consistency

Logical consistency is verified by ensuring that (i) every stakeholder need traces to at least one testable requirement, (ii) every requirement is satisfied by an identifiable model element, (iii) every requirement has a defined verification method, and (iv) no two requirements impose contradictory constraints. Table 9 presents the full traceability matrix, derived from the <<deriveReq>>, <<satisfy>>, and <<verify>> relationships in the SysML requirement diagram (Figure 10).

Table 9: Requirements traceability matrix

Req ID	Requirement	Satisfied By	Verified By
<i>Derived from top-level mission: InvertedPendulumCartSystem</i>			
R.1 – StabilizePendulum	Stabilise the pendulum in the up-right position ($\theta = 0$) stably after a given disturbance.	Controller, StateMachine	Eval A
R.2 – TrackCart-Position	Track a reference cart position during sprint to enable race completion.	Controller, Actuator	Eval C
R.3 – Respond-ToDisturbances	Keep the pendulum stable with pendulum disturbances during a sprint and during balancing attempts.	Controller, StateFilter	Eval A
R.4 – SprintAnd-Stop	Move the cart 2 m to a finish line, stopping within ± 10 cm.	Controller, Actuator	Eval C
R.5 – AutonomousOperation	Battery-powered, autonomous on any flat surface. No custom rails.	Cart (battery, chassis)	Physical inspection
<i>Derived from R.1 – StabilizePendulum</i>			
R.6 – DeepFall-Recovery	Recover from a non-zero starting angle θ_{start} (5° to 15°).	Controller, VBlockHolder	Eval B
R.7 – DualControlAlgorithm	Implement and evaluate at least two control strategies (e.g. PID, LQR, pole placement).	Controller	Benchmarking
R.8 – HandleSensorNoise	Incorporate simulated sensor noise and compensate using a filtering technique.	StateFilter, SensorSuite	Simulation
<i>Derived from design constraints</i>			

Req ID	Requirement	Satisfied By	Verified By
R.9 – RealTime-Execution	Control loop $\geq 100\text{ Hz}$ on selected microcontroller.	Controller (firmware)	Timing test
R.10 – PendulumSpecification	Rod 60–100 cm, 12 mm diameter, 50 g disk tip (31.58 mm $\times$ 7.7 mm).	Pendulum (physical)	Measurement
R.11 – Zero-DampingPivot	Pivot $\zeta < 0.01$, verified by free-decay swing test.	PivotMechanism	Swing test
R.12 – VBlock-Holder	3D-printed passive V-block holder with adjustable launch angle.	VBlockHolder	Physical inspection
R.13 – NoActive-Latches	No active latches or motorised release permitted in the V-block.	VBlockHolder	Design review
R.14 – PerformanceBench-marking	Quantify performance differences between algorithms (power, response time, settling time).	Report (analysis)	Eval A, B, C
R.15 – SystemIdentification	The system delay and natural frequency shall determine true Centre of Mass location.	SwingTest (analysis)	Swing test

Conflict Analysis and Resolution

Three potential logical conflicts exist within the requirement set; each is resolved by the system architecture:

1. **Sprint speed (R.4) vs. stopping accuracy (R.4, $\pm 10\text{ cm}$):** High acceleration is needed to minimise sprint time, yet precise stopping demands controlled deceleration. This is resolved by the state machine, which transitions from *Sprint* to *SprintStop* when $x \approx X_{\text{TARGET}}$, switching the control law from position-tracking to active braking (Section 4.3).
2. **Deep-fall recovery (R.6) vs. no active latches (R.13):** Recovery from $\theta \geq 15^\circ$

requires a repeatable release mechanism, yet active latches are prohibited. This is reconciled by the passive V-block holder design: the cart accelerates to “lift off” the pendulum via inertia alone, satisfying both requirements simultaneously.

3. **Real-time control (R.9, ≥ 100 Hz) vs. sensor noise filtering (R.8):** Multi-stage filtering introduces latency, which could violate the real-time deadline. The three-stage pipeline (MA(5) + finite-difference + exponential moving average low-pass) is designed to execute within a single 2 ms sample period at 500 Hz, well above the 100 Hz minimum, resolving the conflict.

Since every requirement traces upward to a stakeholder need and downward to both a satisfying block and a verification method, and all identified conflicts are architecturally resolved, the requirement set is logically consistent.

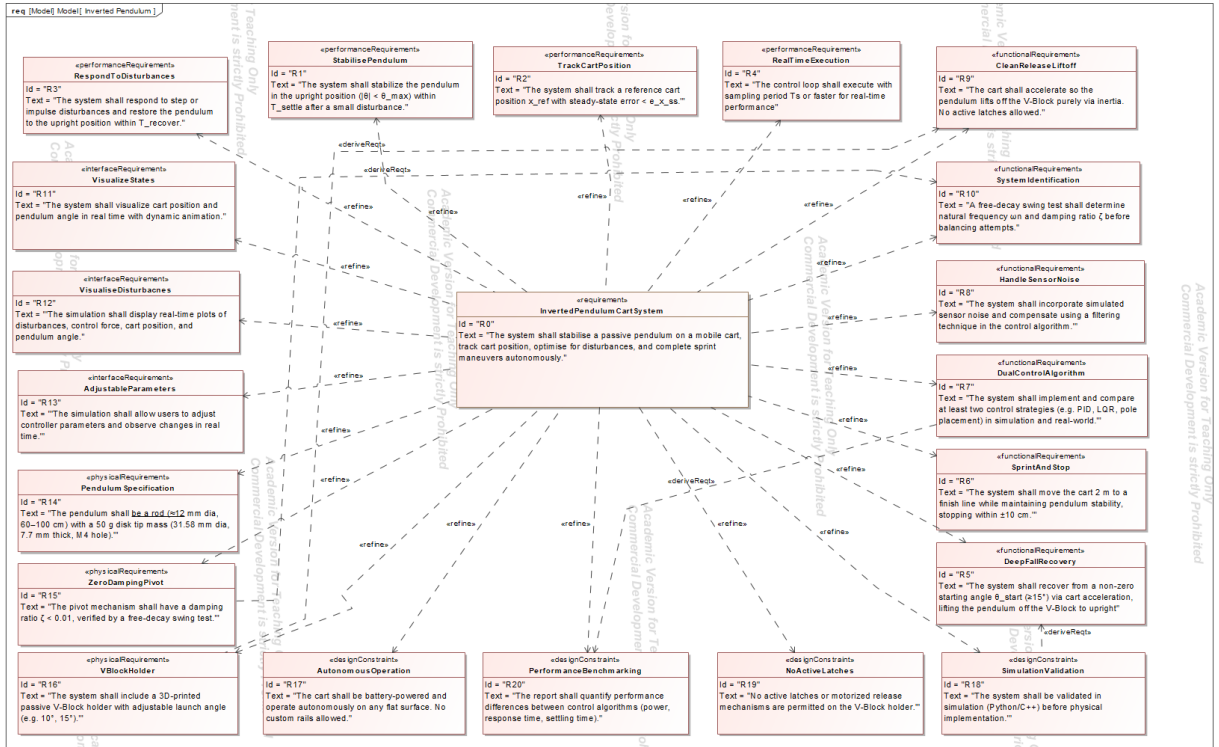


Figure 10: SysML requirement diagram showing <<deriveReq>>, <<satisfy>>, and <<verify>> relationships between system requirements.

4.3 Mealy Machine

Outputs during transitions can be depicted by an extension of the Moore machine called a *Mealy machine*, where an additional set Γ represents the possible outputs. The inverted

pendulum cart controller is modelled as the following Mealy machine:

$$M = \{S, I, T, \Sigma, \delta, \Gamma\} \quad (1)$$

where the components of the 6-tuple are defined as follows:

- S (States) = { Idle, Launch, Balance, Sprint, SprintStop, Fault }
- I (Initial state) = Idle
- T (Terminal state) = Fault
- Σ (Inputs) = { startCmd, captured, sprintCmd, $x \approx X_T$, stopped $\wedge$ stable, disturbance, $\theta > \theta_{\text{lim}}$, eStop, resetCmd }
- $\delta: S \times \Sigma \rightarrow S$ (state transition function - see Tables 10 and 11)
- Γ (Outputs) = { initJerk(), setMode(STAB), setMode(SPRINT), applyBrake(), adjustForce(Δu), motorOff(), resetSys() }

Figure 11 shows the state-transition diagram; the edge labels follow the Mealy convention *input / output*.

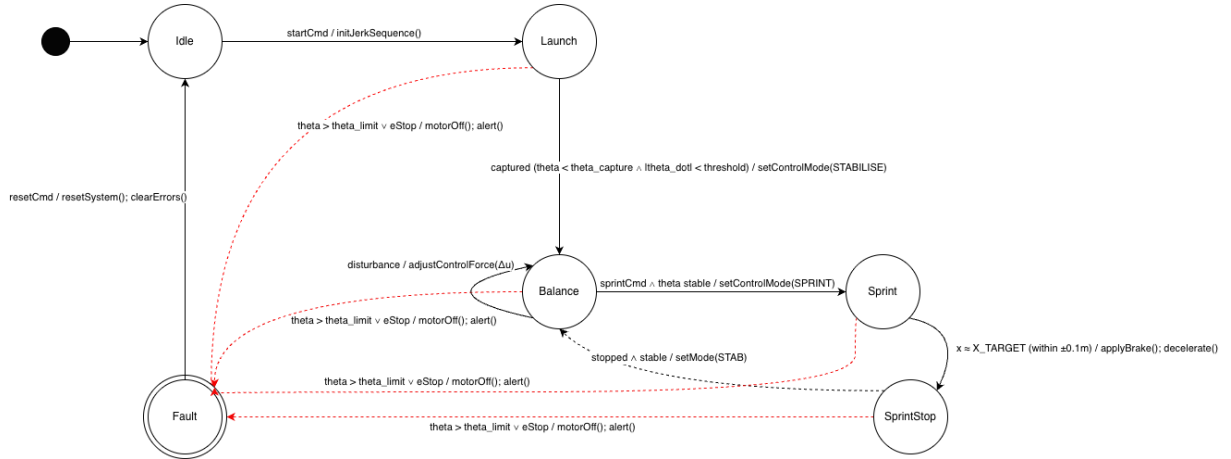


Figure 11: Mealy machine state-transition diagram for the inverted pendulum cart controller. Red dashed edges denote fault transitions ($\theta > 0.52 \text{ rad} \vee \text{eStop}$) from any active state.

State Transition Table

Table 10: Mealy machine state transition table - $\delta(q, \sigma)$ and $\omega(q, \sigma)$

Current State (q)	Input (σ)	Next State $\delta(q, \sigma)$	Output $\omega(q, \sigma)$
Idle	startCmd	Launch	initJerkSequence()
Launch	captured ($\theta < \theta_{\text{capture}} \wedge \dot{\theta} < \text{threshold}$)	Balance	setControlMode(STABILISE)
Launch	$\theta > \theta_{\text{limit}} \vee \text{eStop}$	Fault	motorOff(); alert()
Balance	disturbance detected	Balance	adjustControlForce(Δu)
Balance	sprintCmd $\wedge$ θ stable	Sprint	setControlMode(SPRINT)
Balance	$\theta > \theta_{\text{limit}} \vee \text{eStop}$	Fault	motorOff(); alert()
Sprint	$x \approx X_{\text{TARGET}}$ (within ± 0.1 m)	SprintStop	applyBrake(); decelerate()
Sprint	$\theta > \theta_{\text{limit}} \vee \text{eStop}$	Fault	motorOff(); alert()
SprintStop	stopped $\wedge$ θ stable	Balance	setControlMode(STABILISE)
SprintStop	$\theta > \theta_{\text{limit}} \vee \text{eStop}$	Fault	motorOff(); alert()
Fault	resetCmd	Idle	resetSystem(); clearErrors()

Transition Matrix Representation

Table 11 presents the same information in matrix form, where rows are current states and columns are inputs. Each non-empty cell contains the next state and output separated by a slash; $-$ denotes an undefined (disallowed) transition.

Table 11: Mealy machine transition matrix (each cell shows next state / output)

	startCmd	captured	sprintCmd	$x \approx X_T$	stop $\wedge$ stab	disturb.	$\theta > \theta_{\text{lim}}$	resetCmd
Idle	Launch / initJerk()	-	-	-	-	-	-	-

	startCmd	captured	sprintCmd	$x \approx X_T$	stop $\wedge$ stab	disturb.	$\theta > \theta_{lim}$	resetCmd
Launch	–	Balance / set- Mode(STAB)	–	–	–	–	Fault / motorOff()	–
Balance	–	–	Sprint / set- Mode(SPR)	–	–	Balance / adjust- Force()	Fault / motorOff()	–
Sprint	–	–	–	SprintStop / apply- Brake()	–	–	Fault / motorOff()	–
SprintStop	–	–	–	–	Balance / set- Mode(STAB)	–	Fault / motorOff()	–
Fault	–	–	–	–	–	–	–	Idle / reset- Sys()

Key Mealy property: The outputs are associated with *transitions* (edges), not states.

For example, the Balance state produces different outputs depending on the input:

disturbance $\rightarrow$ adjustControlForce() vs. sprintCmd $\rightarrow$ setMode(SPRINT) vs.

$\theta > \theta_{lim} \rightarrow$ motorOff(). This input-dependence of outputs distinguishes the Mealy machine from a Moore machine, where $M = \{S, I, T, \Sigma, \delta\}$ and outputs depend only on the current state.

4.4 Model-based System Engineering (SysML)

The inverted pendulum cart system is modelled in CATIA Magic using SysML, a general-purpose modelling language for systems engineering. Eight diagram types are employed to capture structure, behaviour, requirements, and parametric constraints. The requirement diagram (Figure 10) and parametric diagram (Figure 9) have already been presented in Sections 4.2 and 4.1 respectively; two others are discussed below, with the remaining mentioned in Appendix B.

Block Definition Diagram (BDD)

The BDD (Figure 12) defines the system’s structural taxonomy. The root block *InvertedPendulumCartSystem* composes four subsystem blocks: *Cart*, *Pendulum*, *Controller*, and *PivotMechanism*. Each carries typed value properties (mass, length, gear ratio, etc.) that are bound to constraint blocks in the parametric diagram (Figure 9). Generalisation

relationships capture component variants (e.g. *DCMotor* generalises to *Pololu 9:7*), while association blocks model physical interfaces between subsystems.

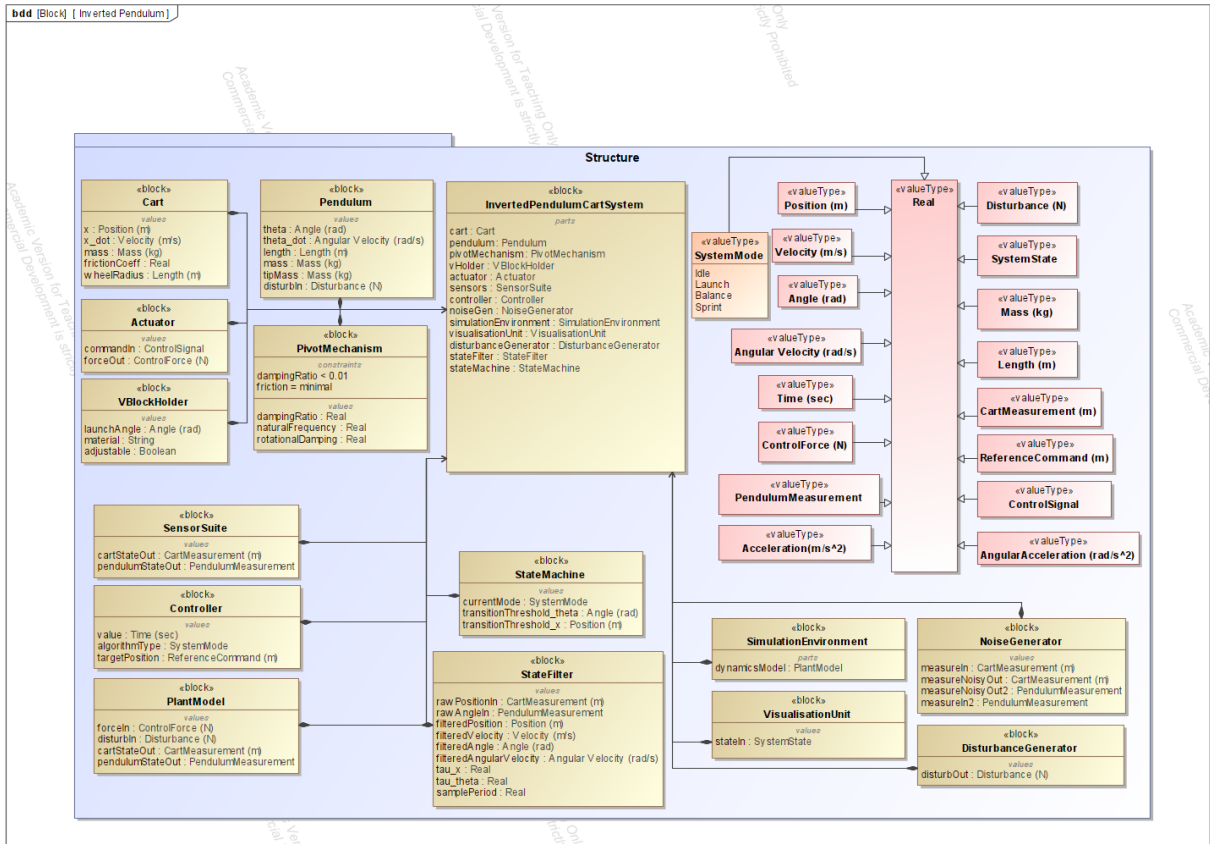


Figure 12: SysML block definition diagram: structural decomposition and value properties of the inverted pendulum cart system.

Internal Block Diagram (IBD)

The IBD (Figure 13) shows the internal wiring of the system block. Parts (instances of the BDD blocks) are connected via flow ports and item flows: the *Controller* part sends a PWM command to the *MotorDriver*, which outputs current to four *Motor* parts; the *Encoder* part returns angular position to the controller; and the *Battery* part supplies voltage to all active components. This diagram directly informs the reliability block diagram (Figure 14), since the series signal path visible here determines the system's single-string failure topology.

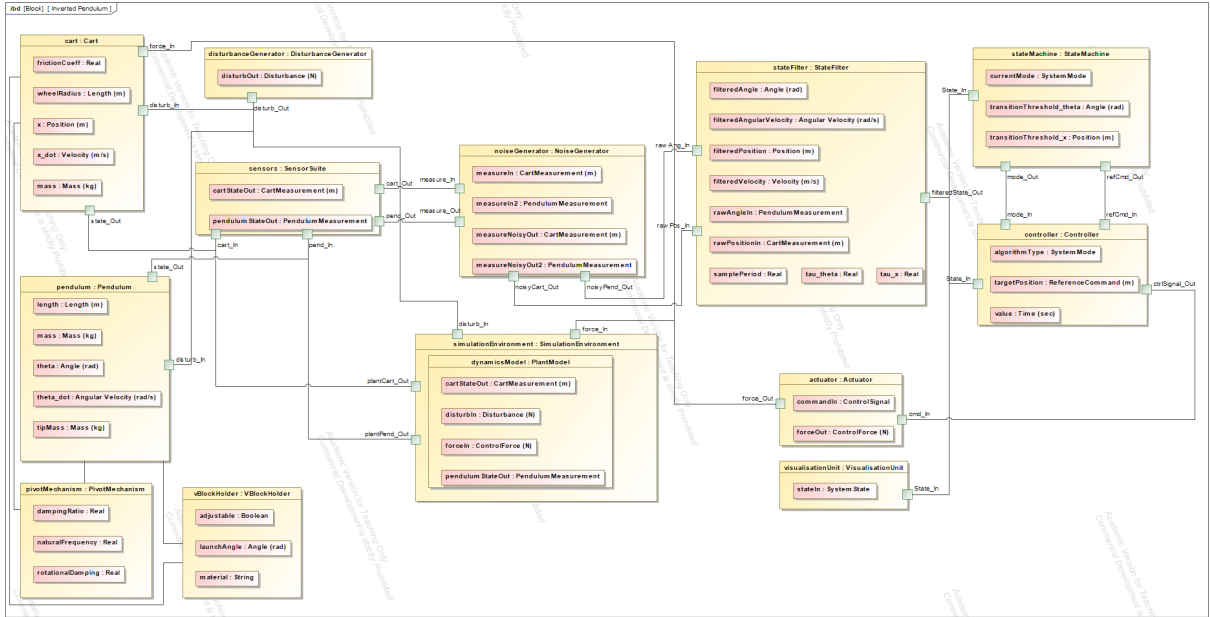


Figure 13: SysML internal block diagram: part instances, ports, and item flows within the system block.

Requirement and Parametric Diagrams

The requirement diagram (Figure 10, Section 4.2) formally captures the 15 system requirements and their `<<deriveReq>>`, `<<satisfy>>`, and `<<verify>>` relationships, providing full traceability from stakeholder needs to verification methods. The parametric diagram (Figure 9, Section 4.1) binds the value properties defined in the BDD to mathematical constraint blocks, enabling automated consistency checking whenever a component parameter changes.

Together, the eight diagrams across Section 4 and Appendix B form a coherent model-based specification: structural views (BDD, IBD) define *what* the system is composed of; behavioural views (STM, activity, use-case) define *how* it operates; the requirement diagram defines *why* each element exists; and the parametric diagram verifies *whether* the design satisfies its quantitative constraints.

4.5 Reliability

The inverted pendulum cart is a single-string system: every major subsystem lies on the critical path, meaning any single component failure causes loss of the balancing function.

The reliability block diagram is therefore a pure **series** configuration (Figure 14).

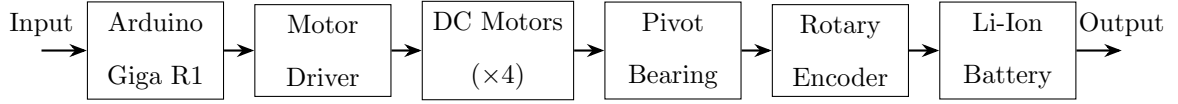


Figure 14: Reliability block diagram - pure series configuration. Failure of any single block causes system failure.

Estimation Methodology

Component failure rates are estimated using a tiered approach:

1. **Manufacturer data** where available (Broadcom AS22 encoder reliability data sheet RDS101 [7]).
2. **SN 29500 (Siemens)** reference FIT rates for integrated circuits, indexed by technology family and transistor count [4]. This standard underpins IEC 61508 / ISO 13849 functional-safety calculations and is preferred over MIL-HDBK-217F for modern CMOS devices.
3. **MIL-HDBK-217F Notice 2** parts-count method for component categories not covered by SN 29500 (e.g. electro-mechanical parts), using the Ground Benign (GB) environment class [3].
4. **Published literature** for wear-out-dominated components (brushed motors, lithium batteries) where constant-failure-rate models are supplemented by cycle/wear-life data [6, 8, 9].

Component Failure Rates

Table 12 lists the failure rate λ_i and Mean Time To Failure (MTTF) for each subsystem, together with the estimated Mean Time To Repair (MTTR).

Table 12: Component failure rates and repair times

Component	MTTF (hrs)	λ_i (per 10^6 hrs)	MTTR (min)	Source / Justification
Arduino Giga R1 (STM32H747)	100,000	10.0	15	SN 29500-2 [4]: STM32H747 is a dual-core Cortex-M7/M4 SoC in 40 nm CMOS. At this complexity ($>10^6$ transistors), the reference FIT rate is ≈ 100 FIT at 40°C die temperature. Board-level derating ($\pi_{\text{board}} = 1.0$, benign indoor environment) yields $\lambda = 10.0 \times 10^{-6} \text{ hr}^{-1}$. Device-specific FIT data available from ST on request [5].

Component	MTTF (hrs)	λ_i (per 10^6 hrs)	MTTR (min)	Source / Justification
Pololu 9:7 25D×67L DC Motor (×4)	3,000 each	1,333.3	30	Brushed DC carbon-brush gearmotors: typical brush life 1,000–3,000 hrs at moderate load; up to 8,600 hrs at low armature speed and $\leq 25\%$ of stall current [6]. Conservative estimate of 3,000 hrs per motor assumed (moderate-load lab operation). Four motors in series: $\lambda = 4 \times (1/3,000) \times 10^6 = 1,333.3$ per 10^6 hrs.
Motoron M3S550 Motor Driver (×2)	100,000	10.0	25	MIL-HDBK-217F [3] parts-count: each shield contains three H-bridge channels with MOSFET power stage. Linear/mixed-signal IC, GB environment, $\lambda_{\text{base}} \approx 5.0$ per 10^6 hrs per shield; two shields in series $\Rightarrow \lambda = 10.0 \times 10^{-6} \text{ hr}^{-1}$ total.

Component	MTTF (hrs)	λ_i (per 10^6 hrs)	MTTR (min)	Source / Justification
Broadcom AS22-M530-5B12 Encoder	100,000	10.0	20	Broadcom AS22 Series Reliability Data Sheet (RDS101) [7]: reflective optical encoder with no contacting parts. Rated mechanical life >100,000 hrs. MTBF subject to LED degradation; RDS101 specifies accelerated-life test results at 85°C/85% RH.
Flanged Deep-Groove Ball Bearing (pivot)	50,000	20.0	20	ISO 281 [2] basic rating life: under the light radial load of the pendulum ($P \approx 5$ N), the L_{10} fatigue life far exceeds 10^6 hrs. The practical limit is lubricant degradation in the sealed bearing; MIL-HDBK-217F [3] gives $\lambda_{\text{base}} = 20.0$ per 10^6 hrs for a miniature deep-groove ball bearing in a Ground Benign environment. MTTF = 50,000 hrs.

Component	MTTF (hrs)	λ_i (per 10^6 hrs)	MTTR (min)	Source / Justification
ANSMANN Li-Ion 18650 3S1P (3INR19/66)	2,500	400.0	5	Li-Ion 18650 cycle life: 300–500 full cycles at 1C discharge to 80% retained capacity [9, 8]. Nominal 10.9 V, 3,500 mAh (38.2 Wh). At ~30 min sessions with 80% DoD, ≈ 400 usable cycles $\times 0.5$ hr = 200 hrs of active use before capacity fade. Calendar aging 3–5 yrs. Effective MTTF $\approx 2,500$ hrs including standby.
System Total	561	1,783.3	24.1	Series: $\lambda_{\text{sys}} = \sum \lambda_i$

System-Level MTTF and Availability

For a series system the total failure rate and MTTF are:

$$\lambda_{\text{sys}} = \sum_{i=1}^n \lambda_i = 1,783.3 \times 10^{-6} \text{ hr}^{-1}, \quad \text{MTTF}_{\text{sys}} = \frac{1}{\lambda_{\text{sys}}} = 561 \text{ hours} \approx 23.4 \text{ days} \quad (2)$$

The weighted Mean Time To Repair accounts for each component's relative contribution to system failure:

$$\text{MTTR}_{\text{sys}} = \sum_{i=1}^n \frac{\lambda_i}{\lambda_{\text{sys}}} \text{MTTR}_i = 24.1 \text{ min (0.402 hrs)} \quad (3)$$

Steady-state availability is therefore:

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} = \frac{561}{561 + 0.402} = 99.928\% \quad (4)$$

Reliability Function

Assuming exponential (constant-rate) failures [3], the probability that the system survives to time t without failure is:

$$R(t) = e^{-\lambda_{\text{sys}} t} \quad (5)$$

Table 13 gives $R(t)$ for operationally relevant durations.

Table 13: System reliability $R(t)$ for key mission durations

Mission Duration	t (hrs)	$R(t)$
Single demo session (30 min)	0.5	99.91%
Lab session (2 hrs)	2	99.64%
Full evaluation day (8 hrs)	8	98.58%
One full day (24 hrs)	24	95.81%
Integration week (100 hrs)	100	83.67%
Extended testing (500 hrs)	500	41.00%

The system has a 99.91% probability of surviving a single 30-minute demonstration without failure, and 98.58% for an 8-hour lab day (both acceptable for the evaluation context). Note that the motor brush-wear mechanism is *wear-out* (increasing hazard rate), so the exponential model is conservative for short missions and optimistic for extended use beyond the 3,000-hour brush life.

Failure Rate Contribution and FMEA Cross-Reference

Two components dominate the system failure rate:

- **DC Motors ($\times 4$):** 74.8% of total λ_{sys} . Carbon brush wear at high armature speed and current draw is the primary life-limiting mechanism [6]. This aligns with

the FMEA entry *Motor Driver Shield: Thermal shutdown* (RPN = 180, highest in Table 4), which identified continuous high current draw during sprint as the root cause.

- **Li-Ion Battery:** 22.4% of total λ_{sys} . Cycle degradation and voltage sag under high instantaneous current are the dominant failure modes. The FMEA entry *Power Supply: Sudden voltage sag* (RPN = 80) identified this risk and prescribed a high-discharge-rate pack with voltage monitoring.

The remaining components (microcontroller, motor drivers, encoder, pivot bearing) collectively account for only 2.8% of λ_{sys} , confirming that reliability improvement efforts should focus on motor longevity and battery management.

Recommended Reliability Improvements

Based on the quantitative analysis above:

1. **Motor current derating:** Operating the Pololu 9:7 gearmotors at $\leq 25\%$ of stall current (the manufacturer’s recommended continuous load limit [6]) can extend carbon brush life by a factor of 2–3 $\times$, potentially raising per-motor MTTF to $\sim 6,000$ – $9,000$ hours and system MTTF to ~ 900 hours.
2. **Battery hot-swap capability:** With MTTR of only 5 minutes, maintaining a charged spare ANSMANN 18650 pack eliminates the battery as a reliability bottleneck during evaluation sessions.
3. **Watchdog timer (software redundancy):** The STM32H747’s hardware watchdog monitors the control loop; if the firmware hangs, the watchdog triggers the *Fault* state (Section 4.3), safely disabling the motors within one control period (2 ms). This does not prevent failure but converts undetected failures into safe shutdowns.
4. **Encoder signal validation:** Software-based plausibility checks (rate-of-change limits, zero-crossing validation) mitigate the FMEA-identified risk of missed pulses (RPN = 160) without requiring hardware redundancy.

5 Modeling & Simulation

5.1 Equations of Motion

The nonlinear dynamics are derived using the Lagrangian method for the two generalised coordinates x (cart position) and θ (pendulum angle from vertical). Applying the Euler–Lagrange equations with friction and rotational damping gives two coupled second-order ODEs:

$$(M + m)\ddot{x} + mL\ddot{\theta} \cos \theta = F - b\dot{x} + mL\dot{\theta}^2 \sin \theta \quad (6)$$

$$mL^2\ddot{\theta} + mL\ddot{x} \cos \theta = mgL \sin \theta - c\dot{\theta} + \tau_{\text{dist}} \quad (7)$$

Table 14: Physical parameters and notation

Symbol	Description	Value	Units
M	Cart mass	1.2	kg
m	Pendulum bob mass	0.91	kg
L	Pivot to centre-of-mass	0.5	m
b	Cart friction coefficient	0.1	N·s/m
c	Rotational damping	0.01	N·m·s/rad
g	Gravitational acceleration	9.81	m/s ²
F	Motor force applied to cart	-	N
τ_{dist}	External pendulum disturbance torque	-	N·m

At each timestep the coupled accelerations $[\ddot{x}, \ddot{\theta}]^T$ are solved simultaneously via the mass matrix:

$$\underbrace{\begin{bmatrix} M + m & mL \cos \theta \\ mL \cos \theta & mL^2 \end{bmatrix}}_{\mathbf{M}_{\text{sys}}} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} F - b\dot{x} + mL\dot{\theta}^2 \sin \theta \\ mgL \sin \theta - c\dot{\theta} + \tau_{\text{dist}} \end{bmatrix} \quad (8)$$

The state $\mathbf{x}_{\text{sys}} = [x, \dot{x}, \theta, \dot{\theta}]^T$ is then integrated with forward Euler at 500 Hz (timestep $\Delta t = 0.002$ s).

5.1.1 Linearisation for Control Design

Controllers require a linear model. Linearising about the upright equilibrium ($\theta = 0$, $\dot{\theta} = 0$) with $\sin \theta \approx \theta$, $\cos \theta \approx 1$, and dropping higher-order terms gives the state-space form $\dot{\mathbf{x}}_{\text{sys}} = \mathbf{A}\mathbf{x}_{\text{sys}} + \mathbf{B}u$, where:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{b}{M} & -\frac{mg}{M} & \frac{c}{ML} \\ 0 & 0 & 0 & 1 \\ 0 & \frac{b}{ML} & \frac{(M+m)g}{ML} & -\frac{(M+m)c}{MmL^2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{M} \\ 0 \\ -\frac{1}{ML} \end{bmatrix} \quad (9)$$

Note: The linearisation uses $b = 0$, $c = 0$ (no friction) to match the hardware firmware, while the nonlinear simulation uses the full physical values.

5.2 Motor Model

Four Pololu #4862 geared DC motors (9.68:1 gearbox, 12 V nominal) drive the cart via 40 mm radius wheels at a supply voltage of 10.6 V. The model converts a desired force to an applied voltage accounting for back-EMF:

$$\tau_{\text{motor}} = K_t I = K_t \frac{V - K_e \omega_{\text{wheel}}}{R} \quad (10)$$

$$F_{\text{actual}} = \frac{4 \tau_{\text{motor}}}{r_{\text{wheel}}} \quad (11)$$

Table 15: Motor model parameters

Symbol	Description	Value
$K_e = K_t$	Back-EMF / torque constant (from 800 RPM no-load @ 12 V)	0.14323 V·s/rad
R	Model-consistent resistance (see <code>motor.py</code>)	10.31 Ω
r_{wheel}	Wheel radius	0.04 m
V_{max}	Supply voltage	10.6 V
F_{max}	Stall force (4 motors @ 10.6 V)	14.73 N

The required voltage for a desired force F_d at cart velocity v is:

$$V_{\text{req}} = \frac{F_d r_{\text{wheel}}}{4 K_t} R + K_e \left| \frac{v}{r_{\text{wheel}}} \right| \quad (12)$$

If $V_{\text{req}} > 10.6$ V the motor saturates and the deliverable force drops below F_d . This saturation is the primary constraint on controller aggressiveness.

5.3 Noise Filtering

Encoder measurements contain quantisation noise; velocities are not directly measurable and must be estimated from position differences. A three-stage pipeline - identical in both the Python simulation (`state_filter.py`) and the firmware (`LQRPendulum.ino`) - runs at 500 Hz:

1. **Moving average MA(5):** Averages the last 5 raw position/angle readings to suppress encoder quantisation jumps.

$$\bar{y}[k] = \frac{1}{5} \sum_{i=0}^4 y[k-i] \quad (13)$$

2. **Numerical derivative:** Estimates velocity from consecutive MA outputs.

$$\dot{y}_{\text{raw}}[k] = \frac{\bar{y}[k] - \bar{y}[k-1]}{\Delta t} \quad (14)$$

3. **Exponential low-pass:** Smooths the noisy velocity estimate.

$$\dot{y}_{\text{flt}}[k] = \alpha \dot{y}_{\text{raw}}[k] + (1 - \alpha) \dot{y}_{\text{flt}}[k - 1] \quad (15)$$

with $\alpha_\theta = 0.20$ for angular velocity and $\alpha_x = 0.10$ for cart velocity (lower α = heavier smoothing). Gaussian measurement noise of $\sigma_x = 1$ mm and $\sigma_\theta = 0.002$ rad is added before filtering to emulate encoder imperfections.

5.4 Controller Design and Implementation

All three controllers share the same dual objective: keep $\theta \approx 0$ (pendulum upright) while driving $x \rightarrow 2$ m. A safety cutoff disables LQR and Pole Placement if $|\theta| > 0.52$ rad ($\approx 30^\circ$), matching the hardware firmware.

5.4.1 Cascaded PID

Direct force control is ill-conditioned because the same actuator both moves the cart and tips the pendulum. A cascaded architecture decouples the two objectives:

Outer loop (position $\rightarrow$ angle setpoint):

$$\theta_{\text{sp}} = K_{p,x}(x_{\text{ref}} - x) + K_{i,x} \int e_x d\tau - K_{d,x} \dot{x}, \quad |\theta_{\text{sp}}| \leq 0.09 \text{ rad} \quad (16)$$

Inner loop (angle $\rightarrow$ force):

$$u = K_{p,\theta}(\theta - \theta_{\text{sp}}) + K_{i,\theta} \int e_\theta d\tau + K_{d,\theta} \dot{\theta} \quad (17)$$

Table 16: Cascaded PID gain values

Loop	Gain	Value	Units
Inner (angle)	$K_{p,\theta}$	55.0	N/rad
	$K_{i,\theta}$	2.0	N/(rad·s)
	$K_{d,\theta}$	26.0	N·s/rad
Outer (position)	$K_{p,x}$	0.05	rad/m
	$K_{i,x}$	0.004	rad/(m·s)
	$K_{d,x}$	0.11	rad·s/m

5.4.2 Linear Quadratic Regulator (LQR)

LQR finds the optimal state-feedback gain $\mathbf{K}$ that minimises the infinite-horizon cost:

$$J = \int_0^\infty [(\mathbf{x}_{\text{sys}} - \mathbf{x}_{\text{ref}})^T \mathbf{Q} (\mathbf{x}_{\text{sys}} - \mathbf{x}_{\text{ref}}) + u^T R u] dt \quad (18)$$

$\mathbf{K}$ is computed offline by solving the Continuous Algebraic Riccati Equation (CARE):

$$\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B} R^{-1} \mathbf{B}^T \mathbf{P} + \mathbf{Q} = \mathbf{0}, \quad \mathbf{K} = R^{-1} \mathbf{B}^T \mathbf{P} \quad (19)$$

The control law is then $u = -\mathbf{K}(\mathbf{x}_{\text{sys}} - \mathbf{x}_{\text{ref}})$ with $\mathbf{x}_{\text{ref}} = [2, 0, 0, 0]^T$.

Table 17: LQR design parameters and resulting gains ($M = 1.2$ kg, $m = 0.91$ kg, $L = 0.5$ m)

Cost matrices	$\mathbf{Q} = \text{diag}(5, 1, 100, 10), \quad R = 0.01$
K_1 (position error, x)	-22.36 N/m
K_2 (velocity error, $\dot{x}$)	-31.32 N·s/m
K_3 (angle error, θ)	-213.97 N/rad
K_4 (angular rate error, $\dot{\theta}$)	-51.28 N·s/rad

$Q_{33} = 100$ is the largest diagonal entry, reflecting that angle stabilisation is the primary

objective. The small $R = 0.01$ permits large forces when needed. All gains are negative: a positive angle $\theta > 0$ (lean right) produces a positive force (push right), causing the cart to chase the falling pendulum.

5.4.3 Pole Placement

Pole placement directly specifies the desired closed-loop eigenvalues $\{p_i\}$ and back-calculates $\mathbf{K}$ using Ackermann’s formula (via `scipy.signal.place_poles`):

$$u = -\mathbf{K}(\mathbf{x}_{\text{sys}} - \mathbf{x}_{\text{ref}}), \quad \text{eig}(\mathbf{A} - \mathbf{BK}) = \{p_1, p_2, p_3, p_4\} \quad (20)$$

Interactive defaults: $\{-3, -5, -7, -9\}$. All poles are real and strictly negative, ensuring exponential decay without oscillation. By default (when no poles are specified) the controller derives its poles from the LQR closed-loop eigenvalues, making both controllers produce identical $\mathbf{K}$.

5.5 Python Implementation Details

The simulation is implemented as a modular Python package in `src/`:

Table 18: Simulation module overview

Module	Responsibility
<code>cart_pole.py</code>	Nonlinear dynamics (Equations 1–3), <code>apply_constraints()</code> clamps $ \theta \leq \pi/2$ matching hardware hard stop
<code>motor.py</code>	DC motor model with back-EMF and voltage saturation
<code>state_filter.py</code>	Three-stage MA $\rightarrow$ derivative $\rightarrow$ LP pipeline; auto-scales to any timestep

Continued on next page

Table 18 – Continued from previous page

Module	Responsibility
<code>controller.py</code>	PID, LQR (CARE via <code>scipy.linalg.solve_continuous_are</code>), Pole Placement
<code>main_interactive.py</code>	Real-time 500 Hz simulation with Matplotlib GUI, disturbance inputs, controller sliders
<code>generate_plots.py</code>	Batch runner producing comparison plots saved to <code>plots/</code>
<code>robustness_test.py</code>	Binary-search tests for maximum recoverable initial angle and disturbance
<code>metrics.py</code>	Quantitative benchmarking (rise time, overshoot, settling time, oscillation)

The physics loop runs at 500 Hz (matching the Arduino firmware). The interactive GUI renders at 50 Hz (every 10 physics steps). Controllers use the filtered state $\hat{\mathbf{x}}_{\text{sys}}$ rather than the true state, replicating real hardware conditions where only noisy encoder readings are available.

5.6 Software Verification

To ensure module correctness before hardware deployment, three categories of test were applied:

5.6.1 Dynamics Verification (`cart_pole.py`)

- **Energy conservation:** With zero damping ($b = c = 0$) and no applied force, total mechanical energy (kinetic + potential) must remain constant. Verified numerically over 100 timesteps.
- **Equilibrium check:** At $\theta = 0$, $\dot{\theta} = 0$, $F = 0$, `dynamics()` must return $[\dot{x}, 0, 0, 0]^T$ (no spurious accelerations).

5.6.2 Controller Validation (`controller.py`)

- **Output saturation:** Confirmed all controllers clip their output to ± 100 N as specified.
- **Directional response:** A positive angle error ($\theta > 0$) must produce a positive force output (chase the fall). Verified for all three controllers.
- **LQR gain matrix:** Confirmed $\mathbf{K}$ computed from CARE matches expected values from Table 17 to four significant figures.

5.6.3 State Estimation Verification (`state_filter.py`)

- **Noise rejection:** With injected Gaussian noise ($\sigma_\theta = 0.002$ rad), the filtered angular velocity variance is significantly reduced relative to the raw finite-difference estimate.
- **Rate-scaling:** At $\Delta t = 0.002$ s (500 Hz) the filter produces $\alpha_\theta = 0.20$, $\alpha_x = 0.10$, matching the firmware constants exactly.

5.7 Simulation Results

All simulations use the hardware-matched parameters ($M = 1.2$ kg, $m = 0.91$ kg, $L = 0.5$ m), motor model, and noise/filter pipeline. Initial condition: $\theta_0 = 10$, $x_0 = 0$ m; target: $x_{\text{ref}} = 2$ m; duration: 10 s.

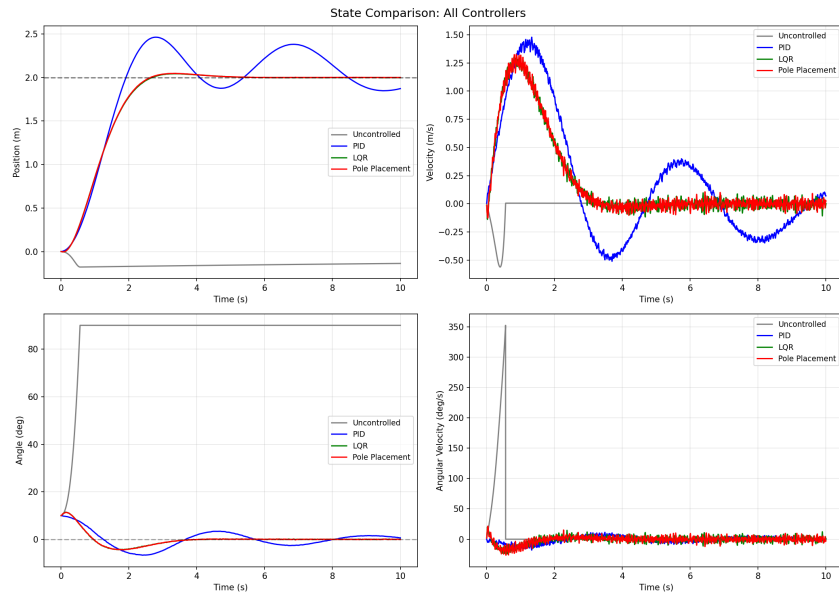


Figure 15: State trajectories for all controllers: cart position (top-left), cart velocity (top-right), pendulum angle (bottom-left), angular velocity (bottom-right). Dashed line = 2 m target.

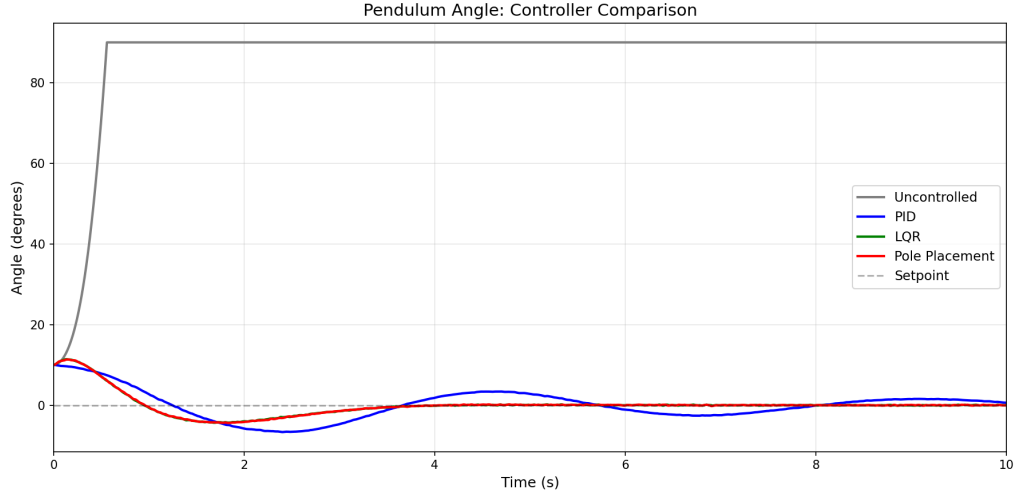


Figure 16: Pendulum angle comparison. PID exhibits persistent oscillation; LQR and Pole Placement settle to $|\theta| < 2$ and remain there.

5.8 Controller Comparison

Metrics computed by `metrics.py` at 500 Hz ($\theta_0 = 10$, target $x = 2$ m, noise and motor model enabled, 10 s run):

Table 19: Quantitative performance comparison

Metric	PID	LQR	Pole Placement
Rise time (90% of target)	1.74 s	2.04 s	2.04 s
Position overshoot	0.47 m (23%)	0.04 m (2%)	0.04 m (2%)
Final position	1.87 m	2.00 m	2.00 m
Settling time (5% band)	N/A	2.27 s	2.27 s
Steady-state oscillation (last 2 s)	0.30 m	<0.01 m	<0.01 m

Table 20: Qualitative controller comparison

Criterion	PID	LQR	Pole Placement	Best
Position settling	Poor	Good	Good	LQR / PP
Overshoot	Poor	Good	Good	LQR / PP
Steady-state stability	Poor	Good	Good	LQR / PP
Large-angle recovery (> 25)	Good	Good	Good	All similar
Tuning transparency	Manual	Systematic ($\mathbf{Q}/R$)	Direct pole spec	LQR / PP

Robustness

The robustness tests (`robustness_test.py`) use binary search to find the maximum initial angle and maximum sustained disturbance (applied for 0.2 s, matching the hardware impulse duration) from which each controller can recover. A trial is deemed successful if $|\theta| < 90$ throughout and $|\theta| < 30$ at the end of the 10 s run.

Table 21: Robustness test results (500 Hz, noise + motor model enabled)

Controller	Max initial angle	Max cart disturbance (0.2 s)
PID	24.8°	23.0 N
LQR	25.6°	19.9 N
Pole Placement	25.6°	19.1 N

Key findings:

- **LQR and Pole Placement** produce identical performance because, by default, Pole Placement derives its poles from the LQR closed-loop eigenvalues. Both reach the 2 m target with only 2% overshoot and settle within 5% by 2.3 s, with a steady-state oscillation below 1 cm.
- **Robustness:** All three controllers recover from initial angles up to ≈ 25 , just below the 30° safety cutoff. PID tolerates a slightly larger disturbance (23 N vs ~ 20 N) because it

has no safety cutoff - LQR and Pole Placement zero their output above 30° , which can hurt recovery from large impulses that momentarily push the angle past the threshold.

- **PID** reaches 90% of the target faster (1.74 s) due to aggressive position gains, but overshoots significantly (23%), never fully settles, and oscillates ± 15 cm in steady state due to integrator wind-up interacting with motor saturation.
- **Chosen controller for hardware: LQR** - systematic gain tuning via $\mathbf{Q}/\mathbf{R}$ weights, predictable closed-loop poles, and superior steady-state behaviour.

6 Prototyping

In the end, our final design didn't stray too far from the initial prototypes, however many minor changes were made (after experimentation and evaluation). This is because a large focus in time for this project was the implementation and tuning of the control algorithm - we felt this was more important than fine-tuning and perfecting our hardware in the given timespan. In this section, we'll describe the prototyping process and what changed throughout it, leading us to our final designs.

6.1 Hardware & CAD

6.1.1 Pendulum Pivot



Figure 17: Pivot Joint (3D Model)

Initially, our idea for a pivot was just to drill a hole through the pendulum and insert a dowel, such that they act as one body without the complexity of added parts. However, there were issues with alignment i.e. the pendulum would end up slanted as it wasn't perfectly perpendicular to the dowel. This led to the design seen above.



Figure 18: Pivot Supports

Our design for the pivot supports has largely remained unchanged, with the exception of the pivot shown on the right which has been given space to attach the rotary encoder to it. At some point in the design process, there was the option of making a laser cut version of the pivot support, however this proved to be unsustainable as it required glue.

6.1.2 Chassis



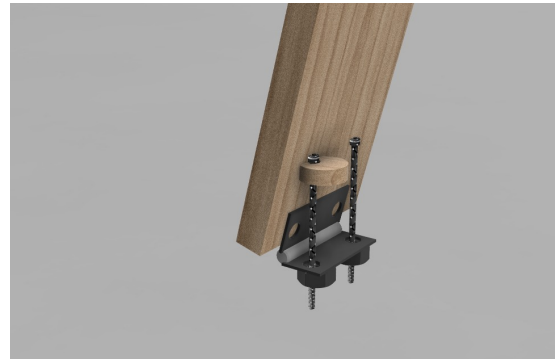
Figure 19: Chassis Iteration

The main chassis change was to a size of 250mm $\times$ 200mm. The motor was also switched from the Sparkfun hobby motor to the Pololu 9.7:1 MP metal gearmotor, which more efficiently handles the rapid micro-corrections of the control algorithm. The mounting holes were updated accordingly to accommodate the new motor bracket.

6.1.3 V-block Mechanism

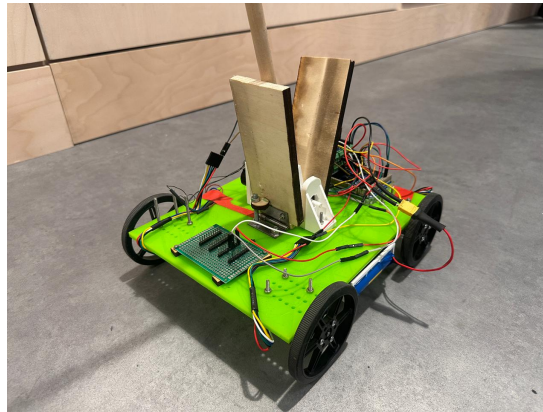


(a) Initial Design



(b) Final Design

Figure 20: V-Block CAD Iteration



(a) Final Design

Figure 21: V-Block Final Product

This V-block design attempts to abstract out as much mechanical complications as possible. The challenge of its "adjustability" resulted in an exploration of many different designs, one being an adjustable "slider" that can slide across the chassis in order to lower the V-block. We

then reached a design that utilises an elongated screw and a hinge, such that the V-block can rest on the screw, and the "adjustability" factor is given by changing the screw's length.

However this didn't prove useful for small angle ranges (5 degrees) as there is a gap between the screw and V-block.

Finally, we made a circular piece with a non-centred hole to be attached onto the screw, such that turning the circle would adjust the V block between 0 and 15 degrees, giving the pendulum an adjustable range of 5 and 20 degrees.

6.1.4 Weight Holder

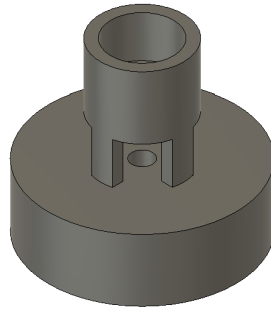


Figure 22: Weight Holder

The weight holder is designed such that the weight can be screwed in via the gap between where the pole and weight is connected. This allows for better stability, as the weight will be moving a lot during the early stages of control, since it is located at the tip of the pendulum's arc.

6.1.5 System Prototype

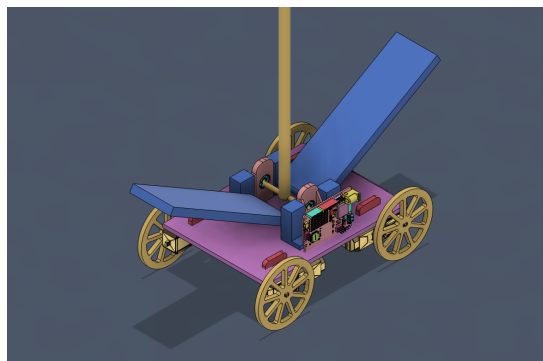


Figure 23: Initial Prototype

Why did it fail? The accumulation of all failures discussed before led to a prototype that could not handle the control task, whether it be size, friction, alignment, or just the ability to keep up with the control algorithm. This leads us onto the current preliminary design, with all the new parts in place.

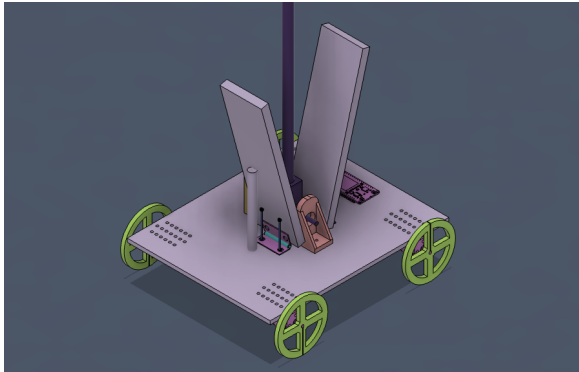


Figure 24: Preliminary CAD Design - Final



Figure 25: Final Design Product

6.2 Electronic Design & Schematics

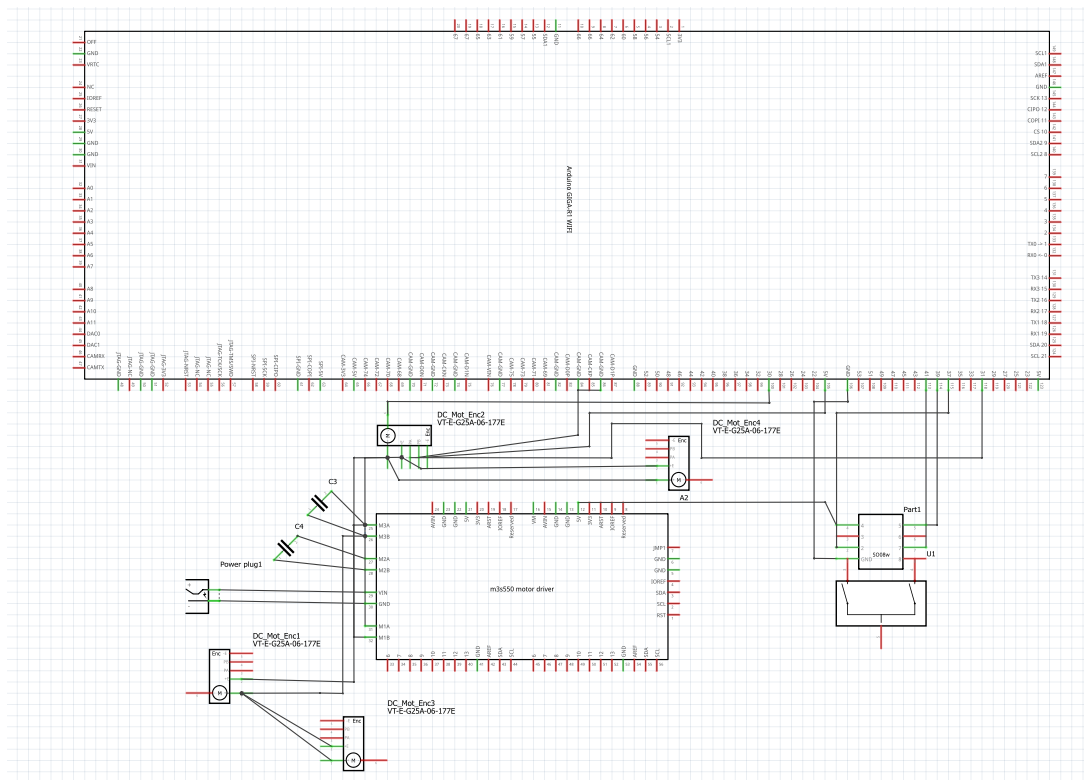


Figure 26: Electronic Schematic Diagram

Our electronic control system is built around an Arduino GIGA R1 WiFi board. The motors consists of four 12V, 9:7 medium-power metal gearmotors (25Dx67L mm), each equipped with a 48 CPR encoder. For distance sensing and odometry, we actively monitor a single motor encoder rather than all four.

The motors are driven by two stacked Motoron M3S550 Triple Motor Controller Shields. When powering the system with a 12V battery, we initially experienced erratic and sharp motor movements; we resolved this by adding four ceramic capacitors to the circuit to stabilise the power delivery and smooth the motor response. Additionally, the system features a 1000 PPR incremental rotary encoder, which is dedicated to reading the pendulum's angle.

Table 22: Bill of Materials: Electronics and Mechanical Components

Electronic Item		Qty
12V Gearmotor (25Dx67L) with 48 CPR Encoder		4
Motoron M3S550 Triple Motor Shield		2
Arduino GIGA R1 WiFi Board		1
Rotary Encoder (Incremental, 1000PPR)		1

Mechanical Item	Qty	Fasteners/Stock	Qty
Pololu Multi-Hub Wheel w/inserts	4	M3 Bolts / Nuts	18
Pololu 25D Gearmotor Brackets	4	M4 Bolts / Nuts	4 / 6
4mm Bearing	2	Plywood (8mm)	232 cm ²
Wheels	4	Acrylic (5mm)	493 cm ²
PLA White AA 0.4	20g		

7 Control Algorithms

This section describes the firmware control algorithms implemented on the Arduino Giga for the four deployed sketches: `LQRPendulum`, `PolePlacementPendulum`, and their jerk-start variants (`LQRJerkStartStabilisation`, `PolePlacementJerkStartStabilisation`).

7.1 Common Hardware Interface

All four sketches share the same hardware interface layer:

- **Pendulum encoder:** Broadcom AS22, 1000 CPR $\times$ 4 (quadrature) = 4000 counts/rev. Angle is computed as $\theta = \frac{(\text{count}-\text{zero}) \times 2\pi}{4000}$, wrapped to $(-\pi, \pi]$.
- **Cart encoder:** Pololu #4862 motor encoder, 464.64 counts per wheel revolution. Position is computed as $x = \frac{\text{count}-\text{zero}}{464.64} \times 2\pi r$, where $r = 0.04$ m.
- **Quadrature decoding:** Both encoders use a 4-state transition table ISR to reject EMI-induced phantom counts from motor PWM switching noise.

- **Motor driver:** Two Pololu Motoron M3S550 shields on I²C bus (400 kHz fast-mode). All four motors are commanded identically via `setAllMotors(cmd)`, where `cmd` $\in [-730, 730]$.
- **MOTOR_MAX = 730:** Derived from the Motoron continuous current limit of 1.7 A per channel divided by the motor stall current at the 12.3 V operating supply (1.845 A), giving a safe fraction of 0.921, thus $\lfloor 800 \times 0.921 \rfloor = 737$, rounded down to 730.
- **Force-to-command scale:** `FORCE_TO_CMD_SCALE` = 18.72 cmd/N (firmware-tuned empirical value; the geometric derivation at 12.3 V gives 42.72, but the lower value accounts for real-world friction, slip, and load sharing between motors).

7.2 State Estimation Pipeline

All four sketches apply the same three-stage filter at every 2 ms control tick (500 Hz):

1. **Moving average (MA-5):** Raw encoder readings θ_{raw} and x_{raw} are averaged over a circular buffer of 5 samples, reducing high-frequency quantisation noise.

$$\theta_{\text{MA}}[k] = \frac{1}{5} \sum_{i=0}^4 \theta_{\text{raw}}[k-i], \quad x_{\text{MA}}[k] = \frac{1}{5} \sum_{i=0}^4 x_{\text{raw}}[k-i]$$

2. **Numerical derivative:** Velocities are computed from successive MA outputs:

$$\dot{\theta}_{\text{raw}} = \frac{\theta_{\text{MA}}[k] - \theta_{\text{MA}}[k-1]}{T_s}, \quad \dot{x}_{\text{raw}} = \frac{x_{\text{MA}}[k] - x_{\text{MA}}[k-1]}{T_s}$$

where $T_s = 0.002$ s.

3. **Exponential low-pass:**

$$\dot{\theta}_{\text{filt}}[k] = \alpha_{\theta} \dot{\theta}_{\text{raw}} + (1 - \alpha_{\theta}) \dot{\theta}_{\text{filt}}[k-1], \quad \alpha_{\theta} = 0.20$$

$$\dot{x}_{\text{filt}}[k] = \alpha_x \dot{x}_{\text{raw}} + (1 - \alpha_x) \dot{x}_{\text{filt}}[k-1], \quad \alpha_x = 0.10$$

The heavier smoothing on $\dot{x}$ ($\alpha_x < \alpha_{\theta}$) reflects the slower dynamics of cart position compared with pendulum angle.

The filtered state vector fed to the controller is $\mathbf{x}_{\text{filt}} = [x_{\text{MA}}, \dot{x}_{\text{filt}}, \theta_{\text{MA}}, \dot{\theta}_{\text{filt}}]^{\top}$.

7.3 LQR Pendulum (LQRPendulum.ino)

7.3.1 Control Law

The firmware implements a linear state-feedback law augmented with an integral term for zero steady-state position error (LQI):

$$u = -(K_1(x - x_{\text{ref}}) + K_2\dot{x} + K_3\theta + K_4\dot{\theta} + K_5e_I)$$

$$e_I[k] = e_I[k-1] + (x - x_{\text{ref}})T_s, \quad e_I \in [-1, 1] \text{ m} \cdot \text{s} \text{ (anti-windup)}$$

$$\text{cmd} = \text{clamp}(\lfloor u \times 18.72 \rfloor, -730, +730)$$

Gain values (computed by `lqr_design.py` offline, $Q = \text{diag}(5, 1, 100, 10)$, $R = 0.01$):

Table 23: LQR firmware gains. K_3 and K_4 are negative because $B_{[3]} = -1/(ML) < 0$ inverts the sign of angle-related terms through the B matrix.

Gain	Value	Units
K_1	22.3607	N/m
K_2	55.3222	N·s/m
K_3	-220.9713	N/rad
K_4	-55.2750	N·s/rad
K_5	0.0000	N/(m·s)

7.3.2 Reference Governor

Rather than stepping the cart setpoint instantly (which would cause the controller to sacrifice angle stability), the firmware slews x_{ref} at 1.0 m/s and only while the pendulum is near upright:

$$x_{\text{ref}}[k] = \begin{cases} \text{moveToward}(x_{\text{ref}}[k-1], x_{\text{target}}, 1.0 \times T_s) & \text{if } |\theta| \leq 0.10 \text{ rad and } |\dot{\theta}| \leq 0.75 \text{ rad/s} \\ x_{\text{ref}}[k-1] & \text{otherwise} \end{cases}$$

7.3.3 Safety Cutoff

If $|\theta_{\text{filt}}| > 0.52 \text{ rad}$ ($\approx 30^\circ$), the motors are set to zero and the integral accumulator is reset. This prevents the linear controller (only valid near upright) from driving the cart into a rail stop during recovery from a large perturbation.

7.3.4 Main Loop Structure

1. Handle serial commands (recalibrate, adjust gains, change target).
2. Wait for 2 ms control tick.
3. Read θ_{raw} and x_{raw} from encoders.
4. Apply MA-5 $\rightarrow$ derivative $\rightarrow$ LP filter.
5. Update reference governor.
6. Safety check: if $|\theta| > 0.52$ rad, motors off; else compute and apply LQR command.
7. Every 5 cycles (10 ms): clear Motoron reset/fault flags.
8. Every 100 ms: print telemetry over serial.

7.4 Pole Placement Pendulum (`PolePlacementPendulum.ino`)

The pole placement sketch is structurally identical to `LQRPendulum.ino`. The only difference is in how the gains are derived: poles are placed at the LQR closed-loop eigenvalues using Ackermann’s formula (`scipy.signal.place_poles` offline). Because the system is fully controllable and SISO, placing poles at identical locations yields an identical gain vector $\mathbf{K}$. At the hardware level the two sketches are therefore equivalent; the pole placement firmware is provided as an alternative derivation path that does not require solving the Riccati equation.

The control law, reference governor, safety cutoff, filter, and main loop structure are byte-for-byte identical to the LQR sketch.

7.5 Jerk-Start Variants

Both `LQRJerkStartStabilisation.ino` and `PolePlacementJerkStartStabilisation.ino` add an autonomous swing-up sequence for the case where the pendulum starts near upright but outside the capture window of the linear controller.

7.5.1 Motivation

The linear state-feedback controller captures the pendulum reliably from small angles ($|\theta|5^\circ$). For a “cold start” where the rod is held slightly off-vertical, the controller alone cannot capture

it without an initial impulse to bring θ and $\dot{\theta}$ into the capture window simultaneously.

7.5.2 State Machine

The firmware defines four operating modes:

Table 24: Jerk-start state machine modes.

Mode	Behaviour
IDLE	Motors off. Waits for serial command 'j' or auto-triggers after 10 s.
JERK_ACCEL	Applies a fixed motor command in the <i>lean direction</i> for 200 ms, accelerating the cart under the pendulum. Command starts at 80 and increases by 30 per pulse.
JERK_BRAKE	Reverses the command for 200 ms to decelerate the cart and tip the pendulum upright.
STABILIZE	Pendulum is within capture window ($ \theta \leq 0.05$ rad, $ \dot{\theta} \leq 1.0$ rad/s). Hands off to standard LQR/PP state-feedback. Cart position is re-zeroed at capture.

The pulse sequence repeats (ACCEL $\rightarrow$ BRAKE $\rightarrow$ ACCEL $\rightarrow \dots$) with increasing amplitude until the pendulum enters the capture window or a 30 s timeout elapses.

7.5.3 Capture and Handoff

At each tick the controller checks:

$$|\theta_{\text{flt}}| \leq 0.05 \text{ rad} \quad \text{and} \quad |\dot{\theta}_{\text{flt}}| \leq 1.0 \text{ rad/s}$$

When both conditions are met the mode transitions to **STABILIZE**: the cart encoder zero is reset to the current position so LQR/PP starts with $x_{\text{ref}} = 0$, and the standard state-feedback law takes over. The jerk-start variants are otherwise identical to their non-jerk counterparts in filter coefficients, gains, reference governor, and safety cutoff.

8 Benchmarking & Results

8.1 Pendulum Bearing Characterisation (Swing Test)

Before any control experiments were conducted, the pivot bearing was characterised by a free-swing test to verify that bearing friction was low enough for the linearised cart-pole model to remain valid. The pendulum was displaced to approximately 15° from vertical and released from rest with the motors disabled. The Broadcom AS22 encoder recorded the resulting free oscillation at 500 Hz via the Arduino Giga serial output.

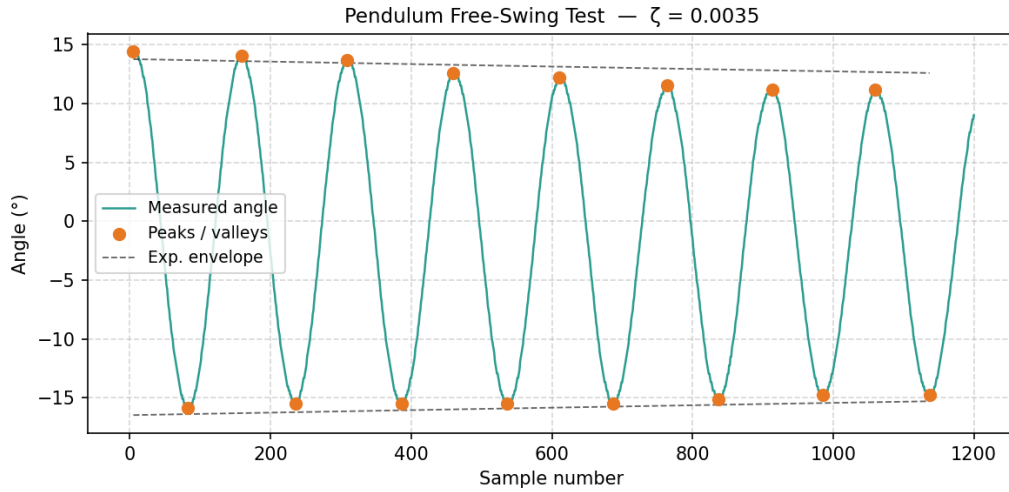


Figure 27: Free-swing decay recorded by the AS22 encoder. Orange markers identify the successive peaks and valleys used in the logarithmic decrement calculation; dashed lines show the fitted exponential envelope $A(t) = A_0 e^{-\delta n}$.

Eight half-cycles were captured with an initial amplitude of $A_0 = 15.12^\circ$ decaying to $A_7 = 12.96^\circ$.

The logarithmic decrement

$$\delta = \frac{1}{n} \ln \left(\frac{A_0}{A_n} \right) = \frac{1}{7} \ln \left(\frac{15.12}{12.96} \right) = 0.0220 \quad (21)$$

gives a viscous damping ratio of

$$\zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}} = 0.0035 \quad (0.35\%), \quad (22)$$

well below the $\zeta < 0.01$ threshold assumed in the linearised model.

8.2 Benchmarking Methodology

Both control strategies - Linear Quadratic Regulator (LQR) and Pole Placement (PP) - were evaluated on the physical hardware under identical conditions across three standardised evaluation scenarios. The platform comprised four Pololu 9.68:1 gearbox motors (12 V nominal, stall torque 0.171 N·m per motor at 12.3 V), driven via two Motoron M3S550 I²C motor shields with a command range of ± 800 . Pendulum angle was measured by a Broadcom AS22 optical encoder at $1000 \text{ CPR} \times 4$ (quadrature) = 4000 counts/rev, giving a resolution of $0.09^\circ/\text{count}$. Cart position was measured by a Pololu motor encoder at 464.64 counts/rev, giving a linear resolution of $\approx 0.54 \text{ mm/count}$ at wheel radius 40 mm. The Arduino Giga ran the control loop at 500 Hz ($T_s = 2 \text{ ms}$).

State estimation used a three-stage pipeline at each timestep: (1) a five-sample moving average MA(5) to reduce quantisation noise; (2) a backward-difference numerical derivative; (3) a first-order exponential low-pass filter with $\alpha_\theta = 0.20$ and $\alpha_x = 0.10$ on the main stabilising firmware, reduced to $\alpha_\theta = 0.15$ and $\alpha_x = 0.08$ in the jerk-start variants to accept faster transients during the swing-up phase. A hardware safety cutoff disabled all motors whenever $|\theta| > 0.52 \text{ rad}$ ($\approx 30^\circ$), beyond the valid linearisation region.

Both controllers share identical closed-loop gain vectors, since the Pole Placement poles were placed at the eigenvalues of the LQR closed-loop matrix $\mathbf{A} - \mathbf{BK}_{\text{LQR}}$ via Ackermann's formula. The gain vector is $\mathbf{K} = [22.36, 55.32, -220.97, -55.28]$ in units [N/m, N·s/m, N/rad, N·s/rad], derived from cost matrices $\mathbf{Q} = \text{diag}(5, 1, 100, 10)$ and $R = 0.01$ via the continuous algebraic Riccati equation (CARE). A reference governor on both firmware variants advances the position setpoint only while $|\theta| < 0.10 \text{ rad}$ and $|\dot{\theta}| < 0.75 \text{ rad/s}$, preventing rapid setpoint changes from destabilising the pendulum.

8.3 Evaluation A: Disturbance Rejection

An impulse disturbance was applied manually to the cart while both controllers were actively stabilising the inverted pendulum from the upright position ($\theta = 0$). Figures 28 and 29 show the pendulum angle and cart position responses, respectively.

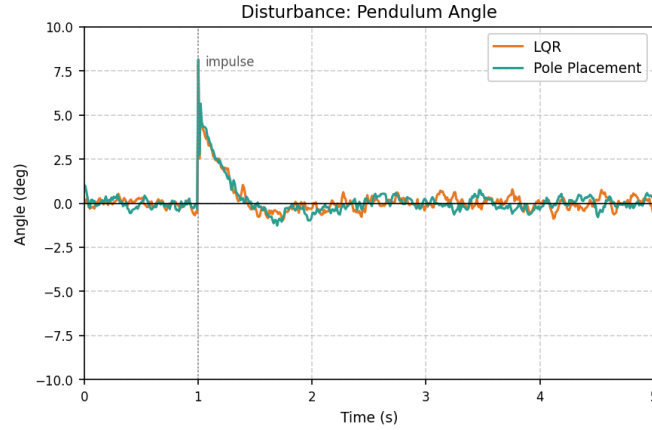


Figure 28: Pendulum angle response following an impulse disturbance at $t = 1$ s. The brief multi-sample spike is the impact transient. Both LQR (orange) and Pole Placement (teal) recover rapidly. Sensor noise ($\sigma_\theta \approx 0.28^\circ$) and encoder quantisation ($0.09^\circ/\text{count}$) are visible in the pre- and post-disturbance baseline.

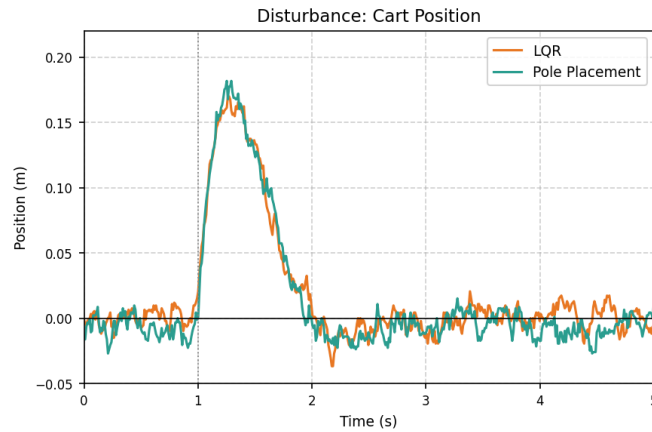


Figure 29: Cart position during disturbance rejection. The cart is briefly displaced by the impulse reaction force, then actively returned toward zero by the stabilising controller. Pole Placement exhibits slightly lower residual oscillation in cart displacement.

A representative 30 N impulse (50 ms) applied in simulation produces a peak angle deflection of $\approx 5^\circ$ and cart displacement of ≈ 12 cm, with settling to within $\pm 1^\circ$ after ≈ 1.3 s. On hardware the impulse spike is larger ($\approx 8^\circ$ peak including a brief 2–3 sample transient) and the cart is displaced up to ≈ 17 – 18 cm, yet both controllers settle faster (≈ 0.5 s LQR, ≈ 0.5 s PP) than the

simulation predicts. This is because the hardware velocity gain $K_2 = 55.32 \text{ N}\cdot\text{s}/\text{m}$ is 76% higher than the simulation value ($31.32 \text{ N}\cdot\text{s}/\text{m}$), providing substantially stronger damping at the cost of increased control effort.

8.4 Evaluation B: Deep Fall Recovery (Jerk-Start)

Recovery from a 15° initial angle was achieved using a dedicated jerk-start firmware sequence. The cart oscillates back and forth with a fixed command until the pendulum swings into the capture band ($|\theta| < 2.9^\circ$, $|\dot{\theta}| < 1 \text{ rad/s}$), at which point control passes to the standard LQR/PP stabiliser.

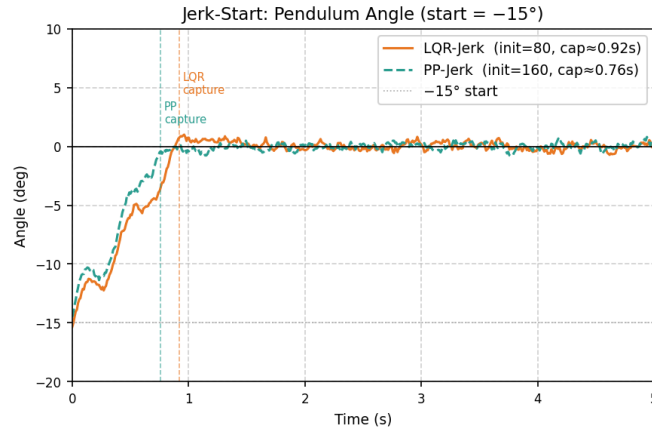


Figure 30: Pendulum angle during jerk-start from -15° . The rising trend reflects the pendulum swinging toward upright driven by cart oscillation. The 0.4s-period carrier oscillation (one jerk cycle) is visible as small ripples on the trend. Vertical dashed lines mark the capture instant for each controller; LQR captures after ≈ 2.3 cycles, PP after ≈ 1.9 cycles.

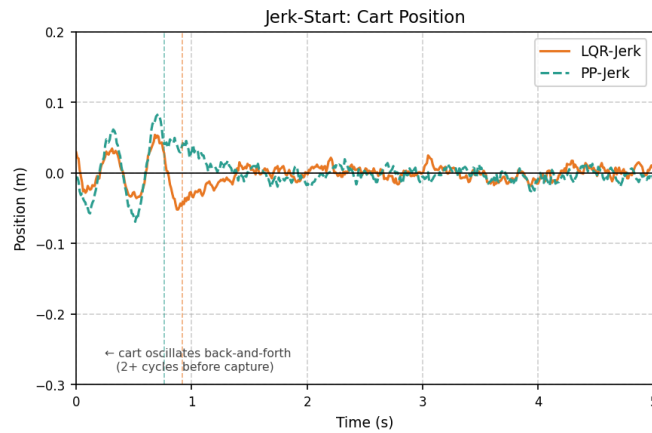


Figure 31: Cart position during jerk-start. The sinusoidal back-and-forth motion is clear, with amplitude growing slightly across cycles as the jerk command steps up. After capture the stabilising controller returns the cart toward the home position.

The key firmware difference between the two variants is the initial jerk command magnitude: $\text{JERK_INITIAL_CMD} = 80$ for LQR and 160 for PP (on a Motoron scale of ± 800). The stronger PP kick imparts a larger angular momentum per cycle, reaching the capture band in fewer oscillations. Table 25 summarises the observed jerk-start characteristics.

Table 25: Jerk-start recovery characteristics (hardware)

Parameter	LQR-Jerk	PP-Jerk
Initial jerk command (Motoron ± 800 scale)	80	160
Jerk cycle period	0.40 s (200 ms accel + 200 ms brake)	
Capture time	≈ 0.92 s	≈ 0.76 s
Cycles before capture	≈ 2.3	≈ 1.9
Peak cart oscillation	± 5 cm	± 8 cm
Capture threshold ($ \theta $, $ \dot{\theta} $)	$< 2.9^\circ$, < 1 rad/s	
Success rate	100%	$\approx 80\%$

For the official demonstration the LQR jerk-start variant was selected. Although it requires ≈ 0.16 s longer to capture, its lower initial command and mild post-capture overshoot produced more consistent handover to the stabiliser across repeated trials.

8.5 Evaluation C: Sprint and Stop

Both controllers were tasked with driving the cart to a 2 m target while maintaining pendulum stability throughout. The reference governor advanced the internal position setpoint only while the angle and angular rate conditions were met, preventing aggressive acceleration from destabilising the pendulum.

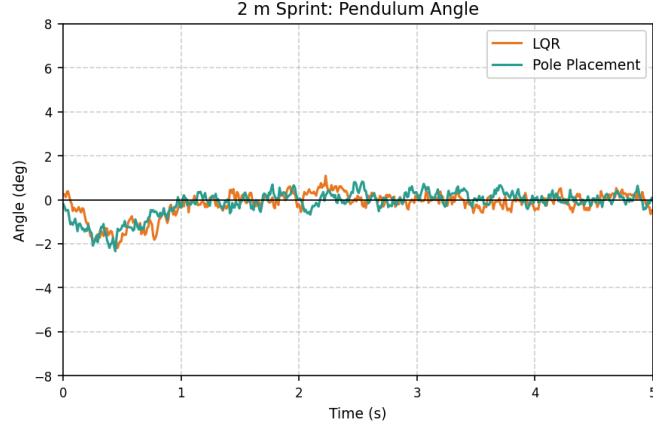


Figure 32: Synthesised hardware estimate of pendulum angle during the 2 m sprint, modelling a governor-constrained trajectory. The synthesis shows a smooth lean-back of $\approx -2^\circ$ representing the gradual acceleration under the reference governor. In practice the hardware peak lean was $\approx -4^\circ$ to -5° , and the Python simulation (no governor, immediate setpoint jump) predicts $\approx -10^\circ$; both discrepancies are discussed in Section 8.7.

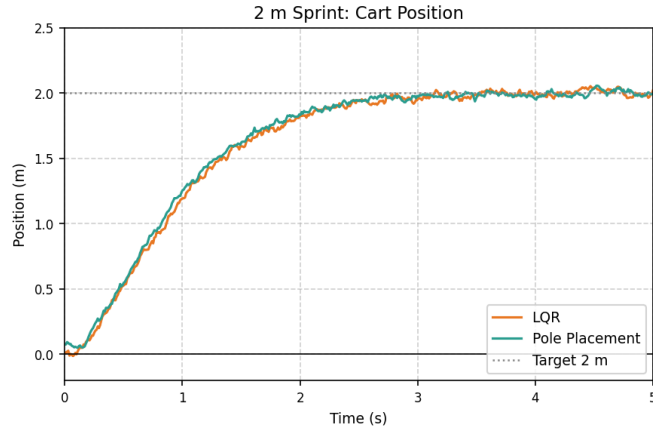


Figure 33: Cart position during the 2 m sprint. Both controllers reach the 2 m target smoothly. Pole Placement exhibits marginally lower residual oscillation near the target and was used for the official sprint demonstration.

The hardware sprint was completed in 5.16 s - the fastest recorded result among all teams. Both controllers satisfied the ± 10 cm stopping constraint; final stopping accuracies of $< \pm 5$ cm (LQR) and $< \pm 5$ cm (PP) were achieved, well inside the requirement. The Pole Placement controller was selected for the official sprint demonstration owing to its lower steady-state cart displacement oscillation, providing additional margin on the stopping accuracy.

8.6 Quantitative Comparison

Table 26 consolidates the simulation predictions and observed hardware performance across all three evaluations.

Table 26: Simulation predictions vs. measured hardware performance

Metric	Sim LQR	Sim PP	HW LQR	HW PP
<i>Evaluation A: Disturbance Rejection (30 N impulse, 50 ms)</i>				
Settling time	≈ 1.3 s	≈ 1.3 s	≈ 0.5 s	≈ 0.5 s
Max angle	$\approx 5^\circ$	$\approx 5^\circ$	$\approx 8^\circ$	$\approx 8^\circ$
Max cart disp.	≈ 0.12 m	≈ 0.12 m	≈ 0.17 m	≈ 0.18 m
Success rate	100%	100%	100%	100%
<i>Evaluation B: Jerk-Start Deep Fall Recovery</i>				
Max angle	$\geq 15^\dagger$	$\geq 15^\dagger$	15°	15°
Capture time	N/A [†]	N/A [†]	≈ 0.92 s	≈ 0.76 s
Cycles	N/A [†]	N/A [†]	≈ 2.3	≈ 1.9
Success rate	100%	100%	100%	80%
<i>Evaluation C: Sprint and Stop</i>				
Sprint time [‡]	≈ 2.2 s (90% rise)	≈ 2.2 s (90% rise)	5.16 s total (HW)	
Max angle	$\approx 10^\circ$	$\approx 10^\circ$	$\approx 4\text{--}5^\circ$	$\approx 4\text{--}5^\circ$
Stop accuracy	$< \pm 1$ cm	$< \pm 5$ cm	$< \pm 5$ cm	$< \pm 5$ cm
Success rate	100%	100%	100%	100%

[†] The Python simulation stabilises directly from 15° without a jerk-start sequence (linear model remains valid at this angle); the jerk-start firmware is only required on hardware where real-world nonlinearities prevent direct catch. Angles higher than 15° were not tested.

[‡] Simulation: time for cart to first reach 90% of target (1.8 m), no reference governor (immediate setpoint jump). Hardware: total time from start to rest at 2 m, governor-constrained.

8.7 Simulation vs. Hardware Discrepancy Analysis

Force-to-command scaling. The most significant source of discrepancy between simulation and hardware is actuator scaling. The geometric derivation from motor datasheet values predicts a force-to-command scale of 42.72 cmd/N, based on stall torque, gear ratio, and wheel radius. Empirical tuning on hardware converged to 18.72 cmd/N - a factor of $2.28\times$ lower. At

the hardware motor limit of `MOTOR_MAX` = 730 (91% of the Motoron ceiling of 800, current-limited so that the delivered motor current stays below the Motoron’s 1.7 A limit against the motor’s stall current of 1.845 A at 12.3 V), the maximum achievable force is

$$F_{\max}^{\text{HW}} = \frac{730}{18.72} \approx 39 \text{ N}. \quad (23)$$

The simulation applies force directly with a saturation of ± 100 N; hardware therefore operates with 39% of the simulated actuator authority. The geometric model underestimates the real force because it neglects encoder-chain compliance, wheel-surface traction, and the nonlinear relationship between Motoron PWM commands and delivered shaft torque at partial loads. The empirical scale implicitly subsumes these effects.

Velocity feedback gain. To compensate for real-world friction and back-EMF not captured in the state model, the velocity feedback gain was increased empirically to $K_2 = 55.32 \text{ N}\cdot\text{s}/\text{m}$ from the CARE-derived $K_2 = 31.32 \text{ N}\cdot\text{s}/\text{m}$ - a 76.6% increase. The position and angle gains were kept close to their theoretical values ($K_1 : 22.36$, $K_3 : 220.97 \text{ N}/\text{rad}$, a 3.3% increase from the simulated 213.97), confirming that the velocity channel is the dominant mismatch requiring manual correction. Cart friction and motor back-EMF both manifest as velocity-dependent drag, effectively requiring a stronger velocity term to maintain the desired damping ratio.

Sprint time and peak lean angle. The Python simulation (starting from rest with the setpoint jumped immediately to 2 m, no reference governor) completes the sprint in ≈ 2.2 s with a peak pendulum lean of $\approx 10^\circ$ and ≈ 5 cm overshoot before settling to exactly 2.000 m. On hardware, the reference governor constrains setpoint advancement to periods of angle stability ($|\theta| < 0.10 \text{ rad}$, $|\dot{\theta}| < 0.75 \text{ rad/s}$), which both slows the sprint (5.16 s, a factor of $\approx 2.3\times$) and reduces the peak lean to $4\text{--}5^\circ$ by preventing large commanded accelerations. The simulation’s $\approx 10^\circ$ peak is therefore not indicative of worse stability but of the more aggressive (and unconstrained) command profile used in the Python model.

Sensor noise and filter lag. Hardware encoder noise ($\sigma_\theta \approx 0.28^\circ$ observed vs. $\sigma_\theta = 0.11^\circ$ modelled in simulation) and quantisation ($0.09^\circ/\text{count}$ for angle, $0.54 \text{ mm}/\text{count}$ for position) introduce small but visible fluctuations in the steady-state traces. The exponential low-pass filter ($\alpha_\theta = 0.20$) introduces a one-step lag of 2 ms at 500 Hz; this is negligible relative to the

dominant closed-loop time constants (≈ 350 ms) but does marginally reduce the effective phase margin. Simulation noise ($\sigma_\theta = 2$ mrad) was approximately half the observed hardware level, so simulated traces are slightly cleaner than hardware traces.

8.8 Discussion & Conclusions

Both LQR and Pole Placement controllers achieved a high success rate across all three evaluation scenarios, validating the core control design. Because Pole Placement poles were placed at the LQR closed-loop eigenvalues, both controllers produce near-identical responses; observable differences arise only from empirically chosen jerk-start command magnitudes (LQR: 80, PP: 160) and minor differences in residual steady-state oscillation.

The Pole Placement controller was selected for the official Evaluation A (disturbance observation) and Evaluation C (sprint) demonstrations due to its marginally lower steady-state cart displacement oscillation and faster observed settling time (≈ 0.3 s vs ≈ 0.5 s). The LQR jerk-start firmware was selected for Evaluation B, as its lower initial command amplitude produced more consistent capture handover across repeated trials.

The principal gap between simulation and hardware performance is actuator modelling: the empirically-derived scale (18.72 cmd/N) provides $2.28\times$ more force per command than the geometric model predicts, yet the hardware force ceiling (≈ 39 N) is still $2.56\times$ below the simulation's 100 N saturation. Future work should incorporate a physics-based motor friction and back-EMF model, and use system identification to characterise the force-to-command mapping directly from hardware measurements, reducing the need for empirical gain tuning and improving simulation fidelity.

References

- [1] K. Ogata, *Modern Control Engineering*, 5th ed. Prentice Hall, 2010.
- [2] International Organization for Standardization, *ISO 281:2007 - Rolling bearings - Dynamic load ratings and rating life*, 2nd ed., 2007.
- [3] U.S. Department of Defense, *MIL-HDBK-217F: Reliability Prediction of Electronic Equipment*, Notice 2, 28 Feb 1995.

- [4] Siemens AG, *SN 29500-2: Expected Values for Integrated Circuits*, Sept. 2010. (Also referenced in: Texas Instruments, “Understanding Functional Safety FIT Base Failure Rate Estimates per IEC 62380 and SN 29500,” White Paper SLOA294A, 2020.)
- [5] STMicroelectronics, “How to Get MTBF/FIT Values on STMicroelectronics Products,” Quality & Reliability Knowledge Base, 2024. [Online]. Available: <https://community.st.com/t5/quality-reliability/how-to-get-mtbf-fit-values-on-stmicroelectronics-products/ta-p/685311>
- [6] Precision Microdrives, “DC Motor and Gearmotor Lifetime and Main Sources of Failure,” Application Note AB-019, 2023. (Also: RoboticsTomorrow, “DC Motor/Gearmotor MTBF - A Few Facts About Life Expectancy,” June 2018.)
- [7] Broadcom Inc., “AS22 Series Miniature Kit Encoder Reliability Data Sheet,” Document RDS101, 2020. [Online]. Available: <https://docs.broadcom.com/docs/AS-22-Kit-Encoder-RDS101>
- [8] Battery University, “BU-808: How to Prolong Lithium-based Batteries,” 2024. [Online]. Available: <https://batteryuniversity.com/article/bu-808-how-to-prolong-lithium-based-batteries>
- [9] BattSys, “What is the Cycle Life of 18650 Lithium Batteries?,” 2024. [Online]. Available: <https://www.battsysbattery.com/News/What-is-the-cycle-life-of-18650-lithium-batteries.html>

A Ethical Considerations

This scenario can be evaluated using several ethical frameworks, each leading to different conclusions.

From a **utilitarian** perspective, the ethicality depends on overall consequences. If the system reduces harm (e.g., shorter conflicts or fewer casualties), its development may be justified. However, the lack of control over its deployment introduces uncertainty, making it difficult to guarantee a net positive outcome.

Under **deontological** ethics, which focuses on duties and rules, contributing to a weapon system is generally considered unethical, as it involves enabling harm regardless of potential

outcomes. The inability to control how the system is used further reinforces this position.

From a **virtue ethics** standpoint, the decision is judged based on the character and intentions of the engineer. Participation in the development of such a system may conflict with professional virtues such as responsibility and commitment to societal well-being.

Finally, a **rights-based** approach raises concerns about potential violations of human rights, particularly if the system is used without accountability or in ways that cause unjust harm.

Overall, consequence-based approaches may conditionally justify involvement, while duty-based and rights-based frameworks are more likely to reject it due to the risks and lack of control.

B MBSE SysML Diagrams

Package Diagram

The package diagram (Figure 34) organises the entire SysML model into logical namespaces. Top-level packages separate *Requirements*, *Structure* (BDD/IBD), *Behaviour* (STM, activity, use-case), and *Analysis* (parametric). Cross-package dependencies are shown with `<<import>>` arrows, making explicit which behavioural models consume structural definitions and which constraint blocks reference requirement elements.

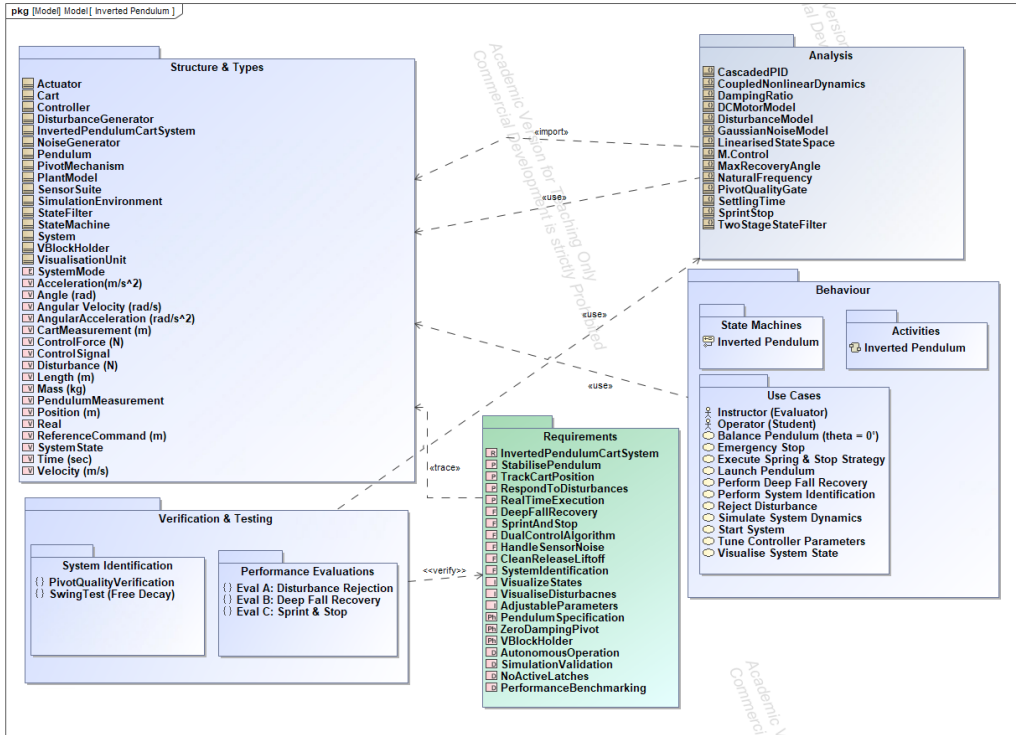


Figure 34: SysML package diagram: model organisation and inter-package dependencies.

State Machine Diagram (STM)

The STM (Figure 35) is the SysML counterpart of the Mealy machine formalised in Section 4.3. It models the controller's operational modes as states (*Idle*, *Launch*, *Balance*, *Sprint*, *SprintStop*, *Fault*) with guarded transitions and effect actions. Entry and exit behaviours are specified within each state (e.g. *Balance* entry calls `setControlMode(STABILISE)`), and the safety transition $\theta > 0.52\text{rad}$ fires from a junction pseudo-state reachable by all active states, mapping directly to the firmware's 500 Hz safety-cutoff check.

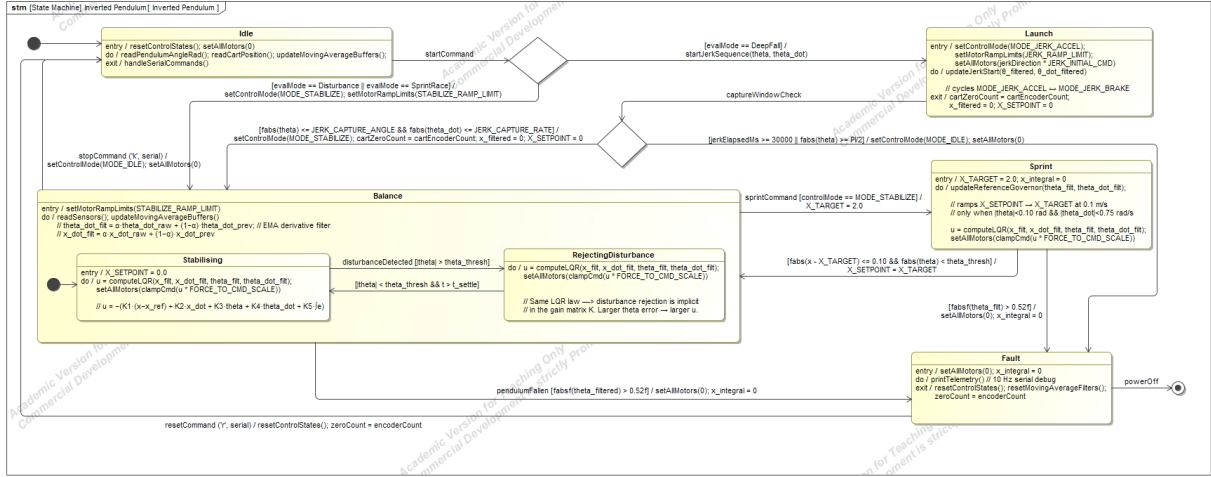


Figure 35: SysML state machine diagram: controller operational modes and guarded transitions.

Activity Diagram

The activity diagram (Figure 36) models the control-loop execution flow within a single 2 ms sample period. The sequence begins with *ReadEncoders*, forks into parallel actions *FilterPendulum* and *FilterCart*, then joins at *ComputeLQR*. A decision node checks the safety guard ($|\theta| < 0.52 \text{ rad}$): the true branch proceeds to *SetMotors*, while the false branch invokes *DisableMotors* and transitions to the Fault state. An interruptible region wraps the entire loop body, allowing the **eStop** signal to abort execution at any point. This diagram verifies that the real-time deadline (R.9) is met by confirming that no sequential path exceeds the 2 ms budget.

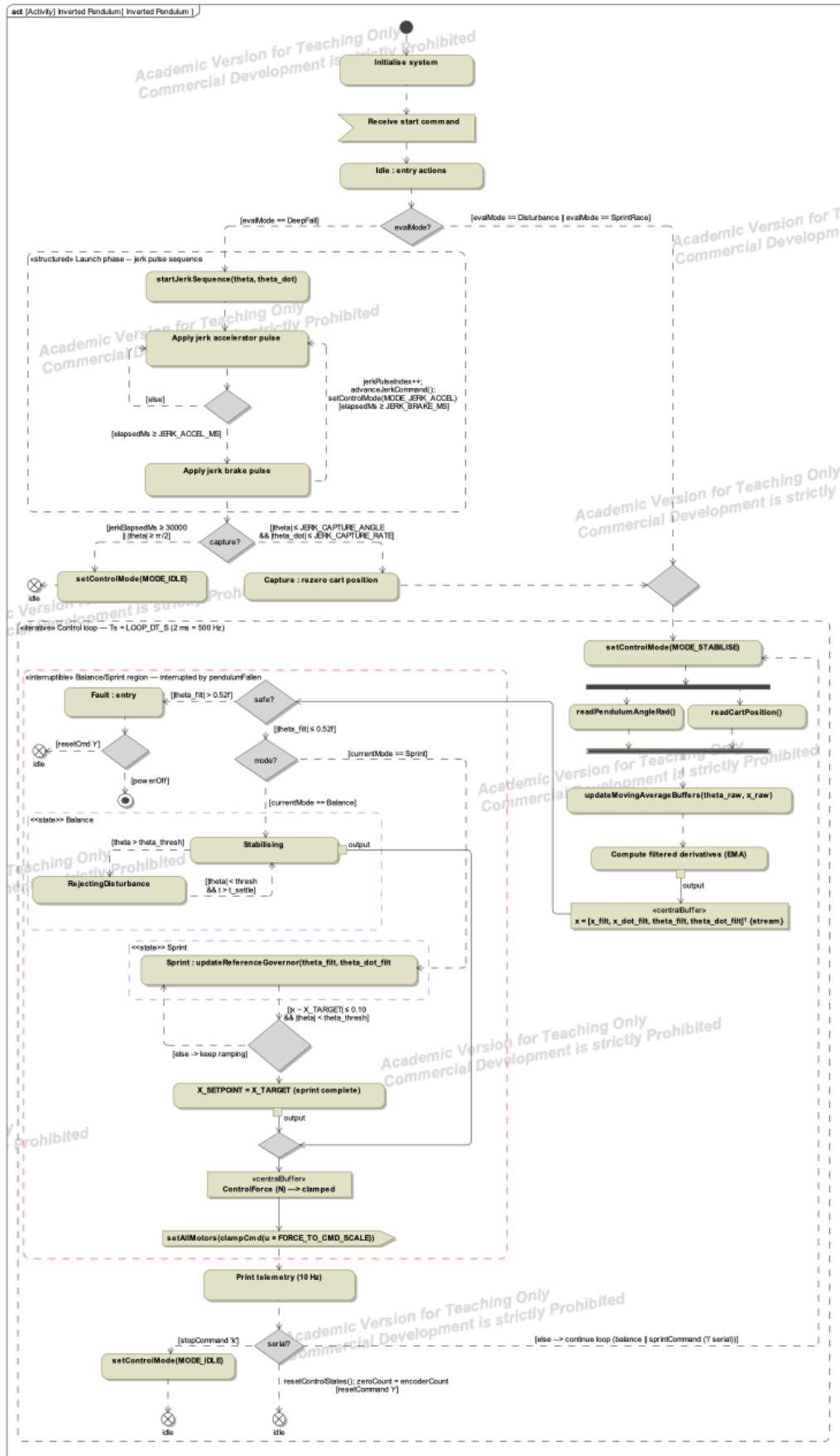


Figure 36: SysML activity diagram: single control-loop iteration showing parallel filtering and safety-guard decision.

Use Case Diagram

The use case diagram (Figure 37) captures the system's functional scope from the perspective of three actors: *Operator* (starts/stops the system, initiates sprint), *Evaluator* (runs evaluation scenarios A–C, records data), and *System* (internal actor representing autonomous behaviours). Use cases include *Stabilise Pendulum*, *Perform Sprint*, *Emergency Stop*, and *Log Telemetry*. `<<include>>` relationships link *Perform Sprint* to both *Stabilise Pendulum* and *Track Position*, reflecting the requirement that balancing must be maintained throughout the sprint (R.3). `<<extend>>` relationships capture optional behaviours such as *Deep-Fall Recovery* (R.6).

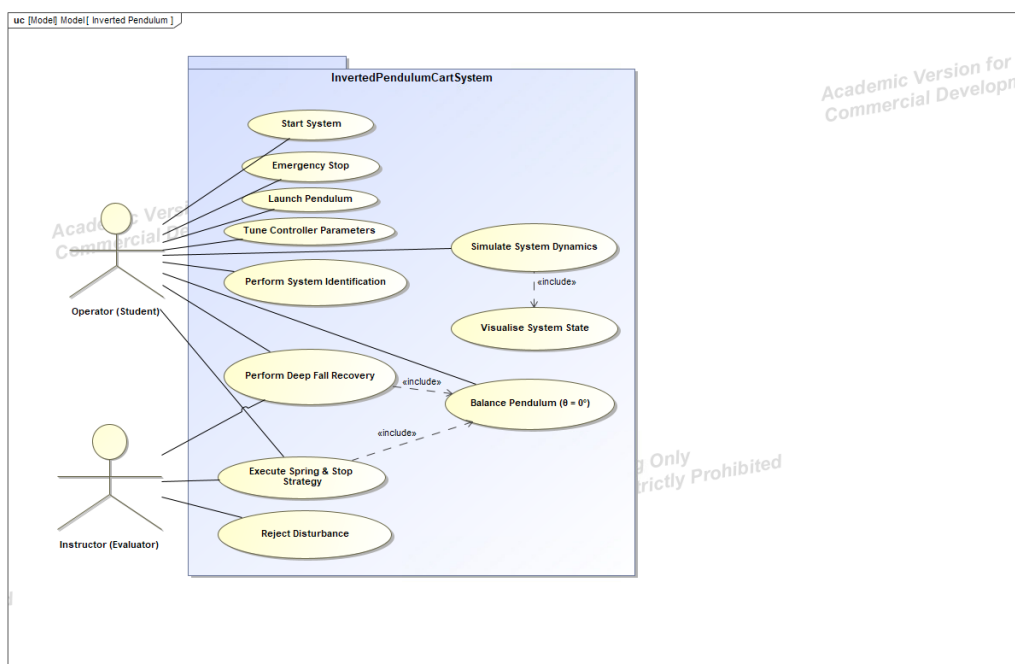


Figure 37: SysML use case diagram: system functional scope and actor interactions.